

# User Requirements Specification – Authentication and Identity Management Service

## 1. Purpose

This document defines the requirements for a **Domain-Driven Design (DDD)**-based Authentication and Identity Management Service. The purpose of this service is to provide a standalone **Identity and Access Management** backend that handles user authentication, authorization data, and session management for applications. It will serve as a reusable **“User Management”** microservice supporting features like user account creation, secure login, role/group assignments, permission enforcement, token-based authentication, and audit trails. The goal is to deliver a flexible and secure authentication service (similar to a mini IAM system) that can be used as a boilerplate for multiple projects <sup>1</sup>. This URS is intended for software developers and outlines both functional and non-functional requirements of the system in a clear, structured manner.

## 2. Scope

The scope of this requirements specification covers all aspects of the authentication and identity domain handled by the service, including user accounts, credentials, roles, groups, permissions, sessions, and related security processes. Specifically, the service will provide:

- **User management and authentication:** User registration, email verification, login with JWT token issuance, password resets, and session handling (including logout).
- **Authorization data management:** Creation and administration of roles and groups, assignment of roles/groups to users, and management of fine-grained permissions attached to groups or roles.
- **Security and audit features:** Secure password storage, token signing/validation, support for external authentication providers (e.g. social logins), and audit logging of important events for compliance and monitoring.

All interactions with this service are intended to occur via a RESTful API (there is no end-user GUI in scope for this backend service). The service acts as a bounded context for identity and access control, meaning other application components or microservices must use its API for authentication/authorization needs rather than directly accessing its data <sup>2</sup>. This document covers requirements for the backend service only; any client application (web or mobile UI) that interfaces with this service is outside the scope of this URS.

**Out of Scope:** Advanced authentication features like multi-factor authentication, biometric login, or behavioral analytics are not included in the initial requirements. Also, integration with enterprise SSO (e.g., SAML, advanced OAuth flows beyond basic external providers) or identity governance processes are beyond the current scope. These can be considered for future enhancements. Additionally, general user profile management (beyond authentication data) is not handled by this service.

### 3. Definitions and Abbreviations

- **Authentication:** The process of verifying a user's identity (e.g., checking username/email and password) <sup>3</sup>.
- **Authorization:** The process of determining what actions or resources an authenticated user is permitted to access (typically via roles or permissions).
- **DDD (Domain-Driven Design):** A software design approach that focuses on modeling the core domain and business logic, using concepts like Entities, Value Objects, Aggregates, Domain Services, and Repositories <sup>4</sup>. In this context, the domain is authentication and identity management.
- **.NET 10:** The target platform/framework for implementing the service (a future or current version of .NET, assumed to be a modern .NET platform for the year 2025).
- **Dapper:** A lightweight object-relational mapper (Micro-ORM) for .NET used for data access. It allows writing SQL queries and mapping them to objects, emphasizing performance and simplicity <sup>5</sup>.
- **JWT (JSON Web Token):** A compact, URL-safe means of representing claims to be transferred between two parties. Here, JWTs are used as cryptographically signed authentication tokens that clients will use to prove their identity on subsequent requests. Each JWT contains claims (such as user ID, roles, etc.) and an expiration time, and is signed by the server's secret key <sup>6</sup>.
- **Token** (in context): Unless otherwise specified, refers to a JWT access token issued to an authenticated user. In other contexts, "token" may refer to one-time tokens like email verification tokens or password reset tokens used to confirm an action.
- **User:** An individual identity account in the system. A user has credentials (like password), profile data (like email), status (active/locked), and is associated with roles, groups, sessions, etc. (See also **Account**; in this document "user" refers to the account, not the person.) <sup>7</sup>.
- **Role:** A named category assigned to users to define broad access level or job function (e.g., *Admin*, *User*, *Guest*). Roles are used for coarse-grained authorization decisions or grouping of users by responsibility <sup>8</sup>.
- **Group:** A collection of users, typically created to assign a set of permissions to all members of that group (e.g., a group representing a department or project team) <sup>9</sup>. Groups support many-to-many relationships with users, and usually have permissions associated with them which group members inherit.
- **Permission:** A fine-grained access right or privilege that can be granted (e.g., *CanResetPasswords*, *ViewReports*). In this system, permissions are not granted directly to individual users but rather through groups (and potentially roles if extended) <sup>10</sup>. A user's effective permissions are determined by the permissions of the groups/roles they belong to.
- **Session:** In this context, a *session* represents an authenticated context for a user, often corresponding to a JWT issued upon login. A session is typically recorded as an entry in the system (with details like user, token identifier, login time, and expiry) to track active logins <sup>11</sup>. Sessions may be used for managing logouts and enforcing token revocation or concurrency limits.
- **Audit Log:** A secure record of important events and changes in the system for traceability and security monitoring <sup>12</sup>. Each audit log entry captures what happened (event type), when (timestamp), by whom (user ID or system actor), and other context (e.g., IP address). Audit logs are append-only (write-only) and preserved for analysis and compliance <sup>13</sup>.
- **External Authentication Provider:** A third-party identity provider that users can authenticate with, such as Google, Facebook, or an enterprise LDAP/OAuth2 server. The service may integrate with these providers so that users can log in using external credentials. An *AuthProvider* record in the domain links a local user account to an external provider account (storing info like provider type and external user ID) <sup>14</sup>.

- **IPasswordHasher**: An interface or component responsible for hashing passwords and verifying password hashes <sup>15</sup>. It abstracts the hashing algorithm (e.g., BCrypt, PBKDF2) and ensures no plaintext passwords are stored or compared directly.
- **ITokenService**: An interface or component for creating and validating tokens (particularly JWTs). It encapsulates the token generation logic (signing, claims inclusion) and hides the details of JWT implementation from the domain logic <sup>16</sup>.
- **IAM**: Stands for *Identity and Access Management*. In this document it refers broadly to the functions of managing user identities, authentication, and access control (roles/permissions). The service described is essentially an IAM service.
- **GDPR**: General Data Protection Regulation, a legal framework in the EU for privacy and data protection. It introduces requirements for handling personal data (such as user emails, IP addresses in logs) – including data minimization, obtaining consent, and allowing users to request data deletion – and imposes rules on data retention <sup>17</sup>. The system's design must facilitate compliance with such regulations where applicable.

*(Any other specialized terms found in this document are explained in context.)*

## 4. Overview of the Authentication Domain (DDD Principles)

The Authentication and Identity Management Service is designed as a distinct **bounded context** in the overall system, dedicated solely to identity and access management concerns <sup>2</sup>. All user accounts, credentials, and authorization-related data live within this context, separate from other domains (e.g., if there is an Orders domain, it will not directly access the user data tables, but instead communicate via this auth service's API). This separation ensures the **auth domain remains cohesive** and that identity logic does not leak into unrelated parts of the system, following DDD's strategic design principles.

**Domain Model and Ubiquitous Language**: In this domain, we clearly define the core concepts – the ubiquitous language includes terms such as *User, Role, Group, Permission, Login, JWT Token, Session, Verification Token*, etc. <sup>18</sup>. These concepts are modeled explicitly in the software as **Entities** or **Value Objects**. For example, a **User** entity represents an account with identity and lifecycle, whereas an **EmailAddress** or **PasswordHash** might be modeled as Value Objects encapsulating validation rules (e.g. proper email format, password hashing) without their own identity. By aligning code structure with real authentication concepts, the system's design mirrors real-world rules and scenarios, making it more intuitive and robust.

### Key Domain-Driven Design Patterns Applied:

- **Entities and Aggregates**: Entities are objects with a distinct identity and lifecycle in the domain (e.g., User, Role, Group). Some entities serve as aggregate roots – for instance, **User** is an aggregate root through which related objects like user credentials, role assignments, sessions, etc., are managed <sup>19</sup> <sup>20</sup>. The User aggregate encapsulates behaviors like verifying a password or managing the user's roles. Other aggregates include **Group** (with its list of permissions) and possibly **Role**, though roles in this design are relatively simple reference data. Each aggregate ensures internal consistency of its invariants; for example, a User aggregate might enforce that no duplicate roles are added to the user, and a Group aggregate might manage adding/removing permissions consistently.
- **Value Objects**: Certain attributes are modeled as value objects to provide stronger typing and encapsulate logic. Examples include an EmailAddress VO (ensuring a valid email format and maybe

normalization) and a PasswordHash VO (which holds the hashed password + salt and perhaps knows how to verify a plaintext password). Value objects are immutable and lack identity – they exist to describe properties of entities. Using VOs for things like emails, passwords, token values, etc., helps validate data at the boundaries of the domain and avoid duplication of validation logic <sup>21</sup> <sup>22</sup> .

- **Domain Services:** Some operations don't neatly belong to a single entity. For example, generating a JWT token involves user data and cryptographic signing that spans beyond a single entity's responsibility. For such cases, domain services or application services are used. A **PasswordHasher** service (possibly domain-level) handles hashing and verifying passwords, encapsulating security policy (e.g., using bcrypt with a certain work factor) <sup>15</sup> . Similarly, a **TokenService** can handle creation of JWTs after login <sup>16</sup> . These services are accessible to the domain or application layer, ensuring the logic (like hashing algorithm choices, token signing) is centralized and easily changeable.
- **Repositories:** The infrastructure uses repository interfaces to provide a collection-like abstraction for persisting and retrieving aggregates. For example, **IUserRepository**, **IRoleRepository**, **IGroupRepository**, etc., define methods to add or retrieve entities from the underlying database <sup>23</sup> . Repositories mediate between the domain and data source (here a SQL database via Dapper), so that domain logic is decoupled from database concerns. By coding to repository interfaces, the core domain logic doesn't depend on any specific data access technology. This allows flexibility; for instance, swapping SQL Server for another database or an in-memory store would not require changing the domain layer <sup>24</sup> .

**Layered Architecture and Clean Separation:** The service design follows a layered (aka *clean architecture*) approach. The **domain layer** contains the Entities, Value Objects, and domain logic rules. An **application layer** (application services) orchestrates use cases by invoking domain operations and coordinating between repositories, external services (like email), and domain entities <sup>25</sup> <sup>26</sup> . The **infrastructure layer** implements the technical details such as the database access (Dapper queries), sending emails, or interfacing with JWT libraries. Crucially, dependencies point inward: the domain layer does not depend on infrastructure (following the Dependency Inversion principle), and all infrastructure details are kept modular. This means we can change how something is done (e.g., switch out the JWT library or the email provider) without affecting core business logic, and we could reuse the same domain logic in a different environment (for example, in a monolithic app or a serverless function) with minimal changes <sup>27</sup> .

In summary, this service's design centers on the authentication **domain logic** – enforcing business rules like unique usernames, secure password practices, verified emails, role-permission relationships – while treating technical details (storage, cryptography, communication) as replaceable components. This ensures that as requirements evolve (say, adding OAuth social login or a new permission type), the change impact is localized and the system can be extended without significant rework of existing code (adhering to Open/Closed principle) <sup>28</sup> <sup>29</sup> .

## 5. Functional Requirements

The service shall provide the following key functional capabilities (use cases) in the authentication domain <sup>30</sup> . Each subsection below outlines the requirements for a specific feature area.

## 5.1 User Registration

- **Account Creation:** The system shall allow a new user to register an account by providing required details such as a unique username, a valid email address, and a password. The username and email must be unique across the system (no duplicate accounts with the same username or email) <sup>31</sup> <sup>32</sup> . If a chosen username or email is already in use, the registration attempt is rejected with an appropriate message.
- **Input Validation:** The service shall validate all registration inputs. The email address shall be verified for proper format (and possibly domain allow-lists if applicable), and the password shall be checked against configured strength rules (e.g., minimum length, required character types) before acceptance <sup>33</sup> . Password complexity requirements (like minimum 8 characters, mix of letters/numbers) should be enforceable. If any input fails validation or policy (invalid format or weak password), the registration is not processed and errors are returned.
- **Password Hashing:** Upon registration, the user's password **shall never be stored in plaintext**. The system must hash the password using a strong, industry-standard hashing algorithm (such as bcrypt or PBKDF2, with an appropriate work factor) and generate a unique salt for the user <sup>15</sup> . The resulting password hash (and salt if separate) is what will be stored in the database for authentication. This ensures that even if the database is compromised, original passwords are not exposed <sup>34</sup> .
- **Initial User Status:** A newly registered user account shall be created in an **inactive or unverified state** by default. The account will not be fully active for login until the user's email is confirmed (see Email Verification) or any other activation step is completed. This status (e.g., a flag indicating the email is not yet verified) is stored with the user record <sup>35</sup> . This prevents unverified accounts from being used to log in if policy dictates email verification is required first.
- **Email Verification Token Generation:** As part of the registration process, the system shall create an **Email Verification Token** associated with the new user. This token is a secure random value (e.g., a GUID or cryptographically strong random string) with an expiration time (for example, 24 hours from creation) <sup>36</sup> . The token is stored (e.g., in an EmailVerificationToken repository/table) and will be used to verify the user's email ownership.
- **Verification Email Dispatch:** The system shall send a verification email to the user's provided email address upon registration. This email contains a link or instructions including the verification token. (In implementation, the email sending could be done asynchronously or via an external service, but from the requirements perspective, it's expected that the user receives a message to verify their account) <sup>37</sup> . If the system does not have an email-sending capability built-in initially, it should be designed such that integrating an email delivery service is straightforward when available.
- **Transactional Integrity:** The creation of the user and the generation of the verification token shall occur in a single logical transaction. The system must ensure that either **both** the user record and token are saved successfully or neither is, to avoid situations like sending a verification email for a user that wasn't created, or creating a user without a corresponding verification token <sup>38</sup> . In other words, the registration process should be atomic.
- **Success Response:** After successful registration initiation, the system shall return a confirmation to the caller (e.g., an API response indicating that the account was created successfully and that verification is required). For security, detailed information should be minimal – e.g., "Registration successful, please check your email to verify your account." The new user's internal ID may be returned if needed by the client, but sensitive details will not be exposed in the response.
- **Audit Logging:** The registration event shall be recorded in the audit log. The system shall create an audit log entry noting that a new user registered (with the user's ID) and include context such as

timestamp and the origin IP or source of registration if available <sup>39</sup> <sup>40</sup> . This provides a security trail for account creations.

## 5.2 Email Verification

- **Verification Process:** The system shall provide a mechanism for users to verify their email address using the token generated at registration (or when an email change is initiated). Typically, this is via an API endpoint that the user accesses by clicking the link in the verification email (e.g., `GET/POST /api/auth/verify-email?token=<token>` ).
- **Token Validation:** When a verification request is received, the service shall locate the corresponding Email Verification Token record using the token value provided. It must verify that the token exists, is associated with a valid user, and has not expired or been used already <sup>41</sup> . If the token is missing, already used, or expired, the verification attempt fails. Expired or invalid tokens should be rejected with a safe generic error (not revealing if the token was ever valid for security).
- **Account Activation:** If the token is valid, the system shall mark the associated user's email as confirmed/verified in the user's profile (e.g., setting a flag `IsEmailVerified = true` ) and, if the account was inactive pending verification, activate the user's account for normal use <sup>41</sup> . Activating the account might involve changing the user's status from "Pending" to "Active". The verification token itself shall then be immediately invalidated — typically by deleting it or marking it as used, so it cannot be reused.
- **Feedback to User:** The service shall indicate the result of verification back to the client. On success, a positive confirmation (e.g., "Email verified successfully, you can now log in") is returned. On failure (invalid or expired token), an error message is returned and, for security, the system may offer to resend a new verification token if within policy, or instruct the user to re-register if applicable.
- **Token Expiry Handling:** The email verification tokens shall have a finite validity (configurable, e.g., 24 or 48 hours). If a user fails to verify within that window, the token is expired and the user's account remains unverified. The system may allow requesting a new verification token (for instance, an endpoint to re-send verification email) as a separate use case (not mandatory in this document, but recommended for usability). In such cases, generating a new token should invalidate the old one.
- **Security Considerations:** The verification link containing the token should be designed to be single-use and hard to guess. The token value must be sufficiently random (at least 128-bit entropy or more) to prevent guessing or forgery <sup>36</sup> . The system should also ensure that the verification endpoint cannot be abused (for example, rate limiting repeated attempts or logging such events for monitoring).
- **Post-Verification Actions:** Once verified, the system could optionally auto-log the user in or send a welcome notification (outside scope of core requirements). The primary requirement is that the user can now authenticate normally. If the system had an *inactive until verified* policy, it should now allow the user to log in after this step.
- **Audit Logging:** Email verification events shall be logged. On successful verification, an audit log entry is added (e.g., event "EMAIL\_VERIFIED" with the user's ID and timestamp) <sup>41</sup> . If there are repeated failed attempts with an invalid token (potentially malicious), the system should log those as well (to the extent possible, though invalid token may not map to a user – it could log an event with the token and IP). This logging ensures there's traceability for account activation and any anomalies in the verification process.

## 5.3 User Login (JWT Generation)

- **Credential Authentication:** The system shall allow a registered user to log in by providing their credentials – typically a username or email along with a password – to an authentication endpoint (e.g., `POST /api/auth/login`). The service will verify the provided credentials against the stored user data. Specifically, it will find the user record by the given username or email (lookup should be case-insensitive as appropriate) <sup>3</sup>. If no matching user is found, or if the password is incorrect, the login shall fail with a generic error (to avoid revealing whether the username/email was valid).
- **Account Status Checks:** When verifying credentials, the system must also check the user's account status. If the user is found but is marked as **inactive**, **unverified**, or **locked** (due to administrative action or too many failed logins), the login attempt is rejected even if the password is correct <sup>3</sup>. For example, if the account requires email verification and it's not done, the system may respond with an error indicating verification is required. If the account is locked for security, it should respond with an appropriate error (and not indicate password validity).
- **Password Verification:** To authenticate, the provided password must be verified against the stored password hash. The system shall use the `IPasswordHasher` (or equivalent) to check the password: e.g., `PasswordHasher.Verify(inputPassword, storedHash)` <sup>42</sup>. This involves hashing the input password with the stored salt using the same algorithm and comparing to the stored hash. This check is done securely (in constant time to prevent timing attacks). No plaintext passwords are ever stored or compared; only the result of the hash verification determines if the password is correct <sup>43</sup>.
- **Failed Authentication Handling:** If the password verification fails or other checks fail (user not found or not allowed to log in), the system shall **not** reveal whether it was the username or password that was wrong. The error message should be generic (e.g., "Invalid credentials") to avoid giving clues. Additionally, the system should record the failed login attempt. It may increment a failure count on the user record and if a threshold of consecutive failures is reached, mark the account as locked to mitigate brute-force attacks <sup>44</sup>. The threshold and lockout duration should be configurable (for instance, lock account after 5 failed attempts within 15 minutes). Each failed attempt, and any account lock action, shall be logged (audit log with event like "LOGIN\_FAIL" and reason).
- **Successful Authentication & JWT Issuance:** If the credentials are verified and the user is allowed to log in, the system shall generate a **JWT access token** for the session. The token generation uses the configured signing key and includes claims about the user's identity and roles/permissions. At minimum, the JWT shall embed the user's unique identifier (user ID) as the subject claim <sup>45</sup>. It may also include the username or email and any relevant authorization claims, such as a list of the user's roles or specific permissions (depending on token size considerations). The token will also contain an issuance timestamp and an expiration timestamp. For example, the default access token lifetime might be 15 minutes or as defined by policy <sup>46</sup>.
- **Secure Token Signing:** The JWT shall be signed using a strong algorithm (e.g., HS256 with a strong secret, or RS256 with an RSA private key). The system's configuration will provide the secret key or certificate required for signing. Only the authentication service (holding the secret key) can generate valid tokens, and consuming clients or services can verify the token's signature to trust its contents. The design abstracts this into an `ITokenService`, so the implementation of JWT signing is centralized and easily maintainable <sup>16</sup> <sup>47</sup>.
- **Session Recording:** Upon successful login, the system shall create a **Session** record representing this login instance <sup>48</sup>. The session record typically contains the user's ID, the token's unique identifier (e.g., JWT ID claim or a generated session ID), the timestamp of login (issued at) and

expiration time of the token. This record is stored in the database (or a cache) and serves multiple purposes: it allows the system or administrators to track active sessions, and it can be used to implement logout or token revocation (by marking a session invalid). Storing the session is especially useful if refresh tokens are used or if we want to blacklist a token before its expiry.

- **Response with Token:** The service shall return the generated JWT token to the client as part of the login response (e.g., in JSON payload). The client will use this token to authenticate subsequent requests to protected services. Along with the token, the response may include token metadata such as expiration time or token type (Bearer). The actual user data in the response should be minimal (for example, possibly the user's ID or display name for client use), since the token itself carries the claims needed.
- **Concurrent Sessions Policy:** By default, the system should allow a user to have multiple sessions (e.g., logged in on multiple devices) unless restricted by policy. Each login generates a new token and session record. If a security policy or config limits concurrent sessions (e.g., maximum N active sessions per user), the system shall enforce that (for instance, by invalidating the oldest session when a new one exceeds the limit, or by disallowing new login until a logout). This is a configurable requirement based on deployment needs (not mandatory by default in this spec).
- **Audit Logging:** Successful login events shall be recorded. The system will create an audit log entry for a successful authentication, including which user logged in and when <sup>49</sup>. Ideally, the log also captures the source IP and perhaps user-agent if available (to facilitate detecting unusual access). Similarly, failed login attempts that result in account locks or are otherwise noteworthy should be logged. This audit trail is critical for security monitoring and forensic analysis.

## 5.4 Session Management

- **Session Tracking:** The system shall maintain a record of active user sessions. Each session corresponds to an issued JWT (or potentially a pair of JWT + refresh token if refresh tokens are introduced). The session data stored includes at least a session identifier (or token identifier), user ID, login time, and token expiration time <sup>11</sup>. This allows the system to know which tokens are currently valid (by issuance) and to manage them (logout, revoke).
- **Logout (Session Termination):** The service shall provide a way for users to log out, which effectively terminates a session. If a user calls a logout endpoint (e.g., `POST /api/auth/logout`) with their token, the system will invalidate that token's session record. This can be done by deleting the session record or marking it as logged out. Once logged out, that token should no longer be accepted as valid if checked. **Note:** Since JWTs are stateless by nature, simply deleting a session record doesn't prevent token reuse unless the service actively checks against the session store on each request. In a simple implementation, logout is mainly handled on the client side (the client discards the token) <sup>50</sup>, but for more secure control, our service is maintaining a session store so that we *can* enforce logout by checking tokens against an allowlist/blacklist.
- **Token Validation (for other services):** While not a direct end-user feature, the authentication service may expose a token validation endpoint (or share public keys for JWT) so that other microservices can verify JWTs. If such an endpoint exists (e.g., `GET /api/auth/validate-token`), it will check the token's signature and expiration, and possibly ensure the token is not revoked (consult the session store). Alternatively, other services can decode JWTs locally using the known signing key, so a validate API may be optional. In either case, token verification is critical for session management – ensuring that any token presented is still valid and not expired or revoked.
- **Session Expiry:** Sessions (and their JWTs) have a built-in expiration. The system shall automatically consider a session invalid once its expiration time passes. There should be a routine or mechanism to clean up expired session records from the database periodically so that the session table doesn't



grow unbounded. For example, a background job could remove sessions that expired, say, more than X days ago. The retention of session history can be tied to audit requirements as well (some might be kept for audit even if expired, or moved to an archive).

- **Multiple Sessions Management:** If a user has multiple active sessions, the system should allow management of those sessions. For instance, an administrator or the user (through a dedicated API) could query active sessions for the account (to list where the user is logged in). They might also forcibly revoke specific sessions (e.g., “logout all other devices”). If in scope, the service shall provide an API for this (e.g., `GET /api/auth/sessions` for current user's sessions, and `DELETE /api/auth/sessions/:id` to revoke one). This is a nice-to-have feature for completeness and security (especially if a token is compromised, the user can kill it).
- **Stateless Scaling Consideration:** The design should enable horizontal scaling; thus, session management is done in a shared data store (the database or a distributed cache) rather than in memory of a single server. The requirement is that any instance of the auth service can handle a logout or validation by looking up the token in the central store if needed. This avoids sticky sessions and allows the service to remain stateless aside from the shared persistence.
- **Performance Note:** Checking every incoming API request's token against a database of sessions could impact performance. As a baseline, the system might not automatically check the session store for each token validation (relying on JWT self-contained validation for speed). However, it must have the capability to do so (for implementing logout or force revoke). In initial operation, the service may trust the token's signature and expiration alone for validating most requests <sup>50</sup>, and only use the session store for explicit operations (logout, admin revocation, audit). This approach can be configurable based on security requirements.
- **Logout All Sessions:** The system should provide a mechanism for users or administrators to invalidate all sessions for a user (for example, if a password is changed or an account is compromised). This would entail deleting all session records for that user and perhaps maintaining a timestamp of last logout so that any JWT issued before that time is rejected. This requirement aligns with security best practices: when critical events happen (account reset, etc.), existing tokens should be curtailed.
- **Audit Logging:** All session-related actions shall be logged. Successful logins (session creation) are logged as mentioned. Explicit logouts should also generate an audit entry (e.g., “USER\_LOGOUT” with user ID and timestamp, possibly which session ended) to have a record of user-initiated or admin-initiated logouts <sup>51</sup>. If an admin revokes someone else's session, that should definitely be audited (including who performed the action). Unusual events like multiple concurrent sessions or tokens being used after logout (which might indicate misuse) could be monitored via the audit logs.

## 5.5 Password Reset

- **Forgot Password Request:** The system shall provide a way for users to reset their password if they have forgotten it or otherwise need to change it without logging in. This is typically initiated by submitting the user's registered email address via an endpoint (e.g., `POST /api/auth/forgot-password` with email). Upon receiving such a request, the system will check if an active user account exists with that email. (For security, the response should be generic regardless of existence, but internally this check is needed.)
- **Password Reset Token Generation:** If the email corresponds to a valid user, the system shall generate a **Password Reset Token** – a secure random token (distinct from the login JWT, typically a one-time secret link) <sup>52</sup>. The token will be associated with that user and have an expiration time (commonly short, like 1 hour or a few hours). If the user had any previous unused reset tokens, those should be invalidated (only one active reset token at a time to avoid confusion and security

issues). The token is stored in a PasswordResetToken repository/table with reference to the user and its expiry time.

- **Reset Email Dispatch:** The system shall send an email to the user's registered email address containing a secure link or instructions to reset their password. The link typically includes the reset token (for example, as a query parameter or part of a URL). The email content should not contain the plain password; it should only guide the user to the secure reset process. If the system doesn't implement actual email sending, it should be designed so that this step can be plugged in easily (similar to verification emails). The existence of this feature must not leak whether the email was found; i.e., even if the email is not in our system, the API should respond with a generic success message like "If that email is registered, a reset link has been sent."
- **Reset Form/Endpoint:** The service shall provide an endpoint for actually performing the password reset when the user clicks the link (for example, a `POST /api/auth/reset-password` that accepts the token and a new password). This could also be done via a web application form that calls the API. The requirements for this step are: the user provides the new desired password (twice for confirmation if through a UI) and the token.
- **Token Validation (Reset):** When a password reset attempt is received, the system shall validate the provided reset token. It looks up the token, ensures it matches an existing token record, has not expired, and is not already used <sup>53</sup>. If the token is invalid or expired, the reset shall be denied. (The system should not allow reuse of a token – once used for a successful reset, it must be marked used or removed.)
- **Password Policy & Update:** If the token is valid, the system next ensures the new password provided meets the required complexity policy (similar checks as in registration). If it doesn't, the reset is rejected with an error indicating password requirements. If it's acceptable, the system shall hash the new password (with a new salt) using the same PasswordHasher mechanism <sup>54</sup>. It then updates the user's stored password hash (and salt) to this new value. This effectively changes the user's password.
- **Token Invalidation:** After a successful password update, the system shall invalidate the reset token (remove it or mark it as used, so it cannot be used again). If someone tries to reuse it, it should be treated as invalid. Also, any other outstanding reset tokens for that user (if multiple could exist due to unusual conditions) should be invalidated to close any loopholes.
- **Related Account Status Changes:** As a side effect, if the user's account was not verified or was inactive, a password reset alone typically would not verify email – those are separate processes. However, the system might assume that if a user can receive a reset email, their email is essentially verified. Some systems choose to auto-verify the email upon a successful password reset as a convenience; this is an implementation choice and can be considered (e.g., mark email verified if they used the reset token) <sup>55</sup>. This URS doesn't strictly require that, but it should be noted if needed by policy.
- **Sessions after Password Change:** (Security consideration) The system **should** invalidate existing active sessions for the user upon a password change as a protective measure. This means if the user (or an attacker) was logged in somewhere with the old credentials, those tokens should ideally be revoked. The requirement is that after a password reset, the user will need to log in again with the new password, and any previously issued JWTs for that account are considered invalid. This can be implemented by deleting all session records for the user at reset time or by using a timestamp field (e.g., record the password last changed time and embed it in token claims to reject old tokens). This ensures an attacker who had a stolen token cannot continue to use it after the password is changed.
- **Feedback to User:** On a successful reset, the service will confirm that the password has been changed (e.g., "Password reset successful, you may now log in with your new password"). On failure

(bad token or policy violation), the user receives an error message (generic for invalid token vs specific for weak password as appropriate). For security, if the token was invalid or expired, the error should not reveal if it was expired vs just wrong, but can suggest re-initiate the reset process.

- **Audit Logging:** The entire password reset flow shall be auditable. A “PASSWORD\_RESET\_REQUEST” event is logged when a reset email is requested (including the user’s ID if the email was found, or at least the email and IP – though if email not in system, it could log just the attempt) <sup>56</sup>. When a password is actually reset successfully, a “PASSWORD\_RESET\_COMPLETE” or similar event is logged with the user ID and timestamp <sup>55</sup>. Additionally, if any security settings changed (like account unlocked or email verified due to reset), those can be noted. These logs help detect potentially malicious resets or simply provide evidence of user account changes.

## 5.6 Role & Group Management

- **Role Definition:** The system shall allow authorized personnel (e.g., administrators) to create, view, update, and delete **Roles** in the system. Each Role consists of a unique name and an optional description describing its purpose <sup>8</sup>. Role names are unique identifiers (e.g., “Admin”, “Manager”, “Customer”) and should be human-readable. The system should enforce uniqueness of role names. Creating or deleting roles likely requires an admin-level privilege and can be exposed via admin API endpoints (e.g., `POST /api/roles`, `DELETE /api/roles/{id}`).
- **Group Definition:** Similarly, the system shall support create/read/update/delete of **Groups**. Each Group has a unique name and description (for example, a group might be “Finance Department” or “Project X Team”) <sup>57</sup>. Groups are a way to collect users for permission management. The system ensures group names are unique to avoid confusion. Group management is also restricted to administrators or authorized users via appropriate endpoints (e.g., `POST /api/groups`).
- **Assigning Users to Roles:** The system shall allow adding or removing role assignments for a given user. For instance, an admin can assign one or more roles to a user account or revoke them. A user can have multiple roles (many-to-many relationship) <sup>58</sup>. When assigning a role, the system should verify the role exists and the user exists. The assignment should be done through the **User** aggregate or a `UserService` to ensure business rules are enforced (e.g., not adding duplicates) <sup>59</sup>. In practice, this might mean a function like `AssignRole(userId, roleId)` which loads the user entity and calls something like `user.AddRole(role)` (internal method) then saves the user. Removing a role would similarly be supported. These operations result in updates to the join table (`UserRole`). The system shall maintain referential integrity – you cannot assign a user to a non-existent role, and you cannot have duplicate entries.
- **Assigning Users to Groups:** Likewise, the system shall allow adding or removing a user to/from a **Group**. A user can belong to multiple groups and a group can have multiple users <sup>60</sup>. The requirement is to provide functionality (e.g., `AddUserToGroup(userId, groupId)` and a remove equivalent) that updates membership. Upon adding, the system might load both the user and group to ensure they exist, then either update via the user aggregate (`user.AddGroup(groupId)`) or group aggregate (`group.AddMember(userId)`) depending on design <sup>61</sup>. Duplicates should be prevented similarly. Removing a user from a group simply means deleting that link.
- **Integrity and Constraints:** When deleting a Role or Group, the system must handle existing relationships. For example, if a role is deleted, what happens to users who had that role? The requirement is either: prevent deletion if in use, or automatically remove that role from all users. The approach should be defined by policy. A safe default: disallow deletion of roles/groups that still have users, or require a cascade removal. This URS expects the design to consider this, but specific behavior can be configured (for now, assume admin will ensure no users depend on it or the system will enforce a check).

- **Role vs Group Usage:** The system distinguishes **Roles** and **Groups** in that roles are typically used to classify users (often tied to job function or high-level access categories), whereas groups are more flexible constructs to bundle permissions and users. In this design, roles do not directly carry permissions by default (so they are more like labels), while groups are the primary way to aggregate permissions <sup>62</sup>. The requirements reflect that: *permissions are assigned to groups (and possibly roles in future if needed)*, but initially roles are not linked to permissions except via group membership. This is an assumption in the model to keep it flexible.
- **Viewing Assignments:** The service shall provide queries to list what roles a given user has, and what groups a user belongs to. Also, list users in a given role or group. This is necessary for admin interfaces. For example, `GET /api/users/{id}/roles` returns roles for that user, `GET /api/groups/{id}/users` returns members of a group, etc. These are read-only operations but important for management.
- **Authorization of Management Actions:** Only users with appropriate administrative privilege (possibly a special role like “Admin”) shall be allowed to manage roles and groups. The system should enforce authorization on these management endpoints. This implies the service itself needs to know the requester’s permissions (likely via JWT claims). Since this is a core auth service, it might use role-based authorization for its own endpoints (for example, only JWTs that contain an “admin” role claim can call the role/group admin APIs).
- **Audit Logging:** All changes to roles and groups must be logged for security and compliance. Creation of a new role or group by an admin is logged (with who created it and the details) as an administrative event. Assignment of a user to a role or group is also logged (who added which user to what role/group, or if the system auto-assigns roles in some flow) <sup>63</sup> <sup>64</sup>. Removal or deletion similarly. These audit logs ensure that there is traceability for privilege changes, which is often a compliance requirement.

## 5.7 Permission Assignments

- **Permission Definition:** The system shall support management of **Permissions** as discrete access rights. Administrators can create new permission entries identified by a unique key/name and description (for example, a permission named “RESET\_PASSWORD” with description “Allows resetting user passwords”) <sup>65</sup>. Each Permission gets a unique ID in the system. Permissions might correspond to fine-grained actions in the application. The service should provide APIs to create, edit, or delete permissions (though in many cases permissions are predefined in code or config, we allow management for flexibility).
- **Assigning Permissions to Groups/Roles:** In the initial design, a **Permission** can be granted to a **Group** (and all users in that group inherit it) <sup>10</sup>. The system shall provide a way to add or remove permission assignments on a group. For example, an admin might call `AssignPermissionToGroup(groupId, permId)` to grant that permission to the group <sup>66</sup>. Internally, the Group aggregate could have a method to add a permission to its collection, enforcing no duplicates and that the permission exists. Removing a permission would similarly be supported (`RevokePermissionFromGroup`). These operations update the join table (GroupPermission) in the database. If the model were extended to allow permissions on roles as well, similar functions would exist for roles (but by default we assume groups are the main way).
- **User Effective Permissions:** The system shall be able to determine a user’s effective permissions based on their group memberships (and potentially roles). Although computing effective permissions at runtime might be more of an application concern, the auth service should offer a function or endpoint like `GET /api/users/{id}/permissions` that returns the list of permissions that user has. It would do so by aggregating all permissions from the groups (and roles,

if roles were to carry permissions) the user is in. This helps other services or admin tools to query what a user can do.

- **Permission Constraints:** The system should ensure consistency such as: you cannot assign a non-existent permission to a group, and permission names must be unique. If a permission is deleted, the system should handle cleanup of that permission from any group assignments (or disallow deletion if in use, depending on policy). Again, a safe approach is to require removal from all groups first or simply cascade delete the join records.
- **Using Permissions in Tokens:** The JWT tokens issued at login *may* include the user's roles or even permissions as claims (especially if needed for client-side or other microservice authorization) <sup>46</sup>. Whether to include a full list of permissions in the token is a design choice (could be large). Alternatively, tokens might just include roles and the application will map roles to permissions. Regardless, the system must maintain the source of truth of these assignments so it can answer permission queries. The requirement is that the design should allow inclusion of at least roles (which then imply certain permissions by how the system is configured) as claims in JWT. If including permissions directly, it should be mindful of token size and update strategy (if a user's permissions change, their existing token might not reflect it until next login or unless using short token lives or a refresh mechanism). This is noted for completeness but the core requirement is the storage and assignment of permissions.
- **Administrative Control:** Only authorized admins can manage permissions and assignments. The service should protect permission management endpoints behind proper authorization checks (e.g., only an "Admin" role user can create a permission or assign it).
- **Audit Logging:** All changes in permission assignments shall be recorded. Granting a permission to a group is a significant event and should be logged (who granted what permission to which group) <sup>66</sup>. Revoking similarly. Creation or deletion of permission definitions should also be logged. Additionally, when a user's effective permissions change (because they were added to or removed from a group, or a permission was added/removed from a group they're in), the event is indirectly covered by those respective logs (group membership change or permission change logs). This ensures that there is a trail for changes in what users can do in the system.

## 5.8 Audit Logging

- **Audit Log Purpose:** The system shall maintain an **Audit Log** of critical actions and events to support security monitoring, troubleshooting, and compliance reporting. The audit log is essentially a chronological record of events such as logins, logouts, account changes, and administrative operations <sup>51</sup>. This helps administrators review what happened in the system (e.g., who logged in when, who changed a user's roles, etc.).
- **Events to Log:** The service must record, at minimum, the following events in the audit log:
- **Authentication events:** Successful logins (with user ID, timestamp, and source IP/device if possible) <sup>49</sup>, failed login attempts (with reason e.g. bad password or locked account), account lockouts (with user and time).
- **Account lifecycle events:** User registrations <sup>40</sup>, email verification successes/failures (tie to user), password reset requests and completions <sup>56</sup> <sup>41</sup>, password changes (if a user changes password while logged in), account deactivation or deletion by admin.
- **Authorization changes:** Creation, modification, or deletion of roles/groups/permissions; assignment or revocation of a role or group to a user; changes in user status (e.g., user promoted to admin role, or removed from a group). For each, log who performed the change (if applicable) and what was changed.

- **Session events:** Explicit user logouts or session terminations by admin, and possibly abnormal events like token invalidation due to security policy.
- **External provider events:** Linking or unlinking of an external auth provider to an account, external login attempts (success/failure).
- **Audit Record Contents:** Each audit log entry shall include: a timestamp, an event type (a short code or description for the action), the **actor** (who performed the action – this could be a user ID if an authenticated user did it, or “SYSTEM” for automated events), the **target** (which entity was affected – e.g., user ID that was modified or logged in), and contextual data (IP address, user-agent for logins, details like “added role X to user Y” or “login failure due to wrong password”). There can be a generic metadata field for additional info. For example, a successful login event might be:  
`Timestamp=2025-11-17T10:30Z, Event=LOGIN_SUCCESS, User=12345, IP=203.0.113.5`.  
 A role assignment event might be: `Event=ASSIGN_ROLE, Actor=adminUserId, Target=userId, Details=Added role "Manager"` <sup>63</sup>.
- **Immutability:** Audit log entries should be **append-only**. The system shall not allow audit records to be altered or deleted via the normal application flow, except as part of a controlled archival/retention process. This is to ensure integrity – once an event is logged, it’s a permanent record. If a correction is needed, a new log entry should be made rather than editing old ones.
- **Storage and Performance:** The audit logs may be stored in a dedicated table ( `AuditLog` ) in the database. The design expects a high volume of writes here, so it should be optimized (e.g., this table can be write-heavy). The system can handle audit writes asynchronously if needed (not required, but an optimization). The requirement is that logging an event should not significantly degrade the performance of the main operation. For instance, if an audit log insertion fails or is slow, it should ideally not prevent the main transaction from succeeding – the system might catch exceptions from logging and proceed, perhaps with an alternate logging mechanism if needed <sup>67</sup>. Reliability of audit logging is important, but it must be balanced with not breaking core functionality.
- **Access and Querying:** The system shall provide a means for authorized users (e.g., security admins) to retrieve and review audit logs. This could be as simple as allowing queries by user or date (e.g., `GET /api/audit-logs?userId=123` or similar), or dumping logs to an external SIEM system. For the URS, at least an API or direct DB access for audit logs is assumed. Only privileged accounts can access audit logs because they contain sensitive info (like IP addresses, which are personal data per GDPR, and information about system usage). The system should also ensure these logs are protected from tampering at the database level (at least by restricting who can delete them).
- **Retention Policy:** Audit logs should be kept in accordance with a retention policy (see Compliance section for specifics). The requirement is that audit logs are retained for a significant period (e.g., 1 year or whatever regulation/business needs dictate) and then disposed of or archived securely <sup>68</sup> <sup>17</sup>. The system should allow configuring this retention period and have a procedure for archiving or purging old log records.
- **Privacy Considerations:** Since audit logs can contain personal data (user IDs, emails, IP addresses, etc.), their management must consider privacy laws. For instance, IP addresses and emails in logs are considered personal data under GDPR <sup>69</sup> <sup>70</sup>. The system might need to anonymize or hash certain fields in logs if required, or ensure logs are only used for their intended purpose (security/audit) and not kept longer than necessary. These considerations should be balanced against the need for comprehensive security records.

## 5.9 External Auth Providers

- **External Login Support:** The system shall be designed to integrate with external authentication providers (sometimes called social login or federated identity). This means a user should be able to

authenticate using an account from an external service such as Google, Facebook, GitHub, or enterprise providers like Azure AD/ADFS or LDAP. While initial implementation may not include all these, the architecture must allow adding them in the future without major refactoring <sup>71</sup>. For example, one might add a “Login with Google” feature that uses OAuth2/OIDC flows.

- **AuthProvider Entity:** The domain model will include an **AuthProvider** (or similar) entity or value object that links a local user to an external provider account <sup>14</sup>. Requirements for this include storing: the provider type (a string or enum like “Google”, “Facebook”, “LDAP”), the provider’s unique user identifier (e.g., Google’s subject ID for that user, or an LDAP DN), and any other relevant metadata (possibly an access token or refresh token from that provider if needed, though ideally not stored long-term). The combination of user and provider is unique – a user can link to a specific external account only once. A user could have multiple AuthProvider links (e.g., link both their Google and Facebook account to the same local user).
- **Account Linking:** The system shall allow an authenticated user to link or unlink an external provider to their account (for example, after logging in with password, they can choose to associate a Google login with their account for future logins). Conversely, if an external login is used and the email matches an existing account, the system could link them (possibly after confirmation to avoid hijacking accounts). The requirements include providing an API to add a link (`POST /api/users/{id}/external-providers`) and remove a link. The linking process should ensure the external account is properly authenticated (for instance, the user goes through the OAuth flow and we receive a valid token from the provider before creating the link).
- **External Login (Sign-In):** The system shall allow users to sign in using an external provider. This typically involves redirecting the user to the provider’s OAuth2 authorization page (handled by the client application), then the provider returns an authorization code which the service can exchange for the user’s details. The requirement on the service side is to handle the callback from the provider: verify the provider’s token, extract the user’s identity (e.g., email or ID), then either find an existing local user mapped to that provider or create a new local user if none exists <sup>71</sup> <sup>72</sup>. For a new user via external login, the service could auto-provision a local account (with a generated username or using their email) and mark it as linked to that provider. Optionally, such accounts could be created in a special “externally registered” status.
- **JWT for External Users:** Once the external provider has verified the user, our service shall issue a JWT for the user just like a normal login. In essence, the external login becomes an alternative means to authenticate, but after that, the user is treated the same in the system (has a user record, roles, permissions, etc.). The JWT should likely include a claim indicating the authentication method (like “auth\_provider”: “Google”) for auditing.
- **Security and Verification:** The system must securely handle external auth data. For OAuth-based providers, it will use the provider’s APIs (requiring client ID/secret configuration in our service). It should verify the tokens or assertions from the provider (e.g., if using OpenID Connect, validate the ID token’s signature and audience). Also, when linking accounts, ensure the user actually controls the external account (via the OAuth dance). If a user attempts to link an external account that is already linked to another user in our system, the operation should be rejected to maintain a one-to-one mapping (unless the design intentionally allows shared external accounts, which is unusual).
- **Extensibility:** A key requirement is that adding support for a new external provider should be low-impact on the system’s core. Thanks to the DDD design, introducing a new provider might involve adding a new implementation of an **ExternalAuthService** or similar strategy for that provider, and maybe extending the AuthProvider types, but it should not require changes in unrelated parts <sup>71</sup> <sup>29</sup>. For instance, our domain already anticipates external providers by including the AuthProvider

link, so integrating, say, a new SAML provider means plugging into that existing model, not rewriting user management.

- **User Experience (Out of Scope):** While UI is out of scope, it's expected that external login flows will be initiated by client applications (web/mobile) using standard protocols. The URS focuses on how the backend supports it: through endpoints to handle callbacks and issuance of JWTs.
- **Administrative Controls:** Admins should be able to see and manage linked external accounts for a user. For example, an admin might query which external providers a user has linked (`GET /api/users/{id}/external-providers`) and possibly remove a link (if a user reported a lost social account or for security). Removing a link does not delete the user, it just disassociates the external login.
- **Audit Logging:** All external authentication events shall be logged. For instance, a successful login via Google is logged as a `LOGIN_SUCCESS` event with provider detail, or a distinct event type like `"LOGIN_GOOGLE_SUCCESS"` could be used. Linking or unlinking an external account to a user should be logged with who did it (user or admin) and which provider was linked <sup>71</sup>. If an external login fails (e.g., token validation fails or the user denied the request), that can be logged as well for debugging and security review. This ensures visibility into the use of external authentication in the system.
- **External Dependencies:** The system's configuration will need to store credentials for external providers (API keys, secrets). The requirement is that these are stored securely (not in code, but in config or a secrets vault) and can be updated without code changes. Additionally, any external provider service outage or slow response should not crash our system; timeouts and error handling around external auth flows must be implemented (e.g., if Google is down, our login via Google should fail gracefully and not affect internal logins).

By fulfilling the above functional requirements, the Authentication and Identity Management Service will cover all core use cases expected in a modern auth system, from user onboarding to daily login and privilege management <sup>30</sup>. The next sections outline non-functional requirements that ensure the service is performant, secure, and maintainable in production.

## 6. Non-Functional Requirements

In addition to the functional capabilities, the service must meet various non-functional criteria to be effective in a real-world environment. These include performance, scalability, security hardening, high availability, proper logging/monitoring, and compliance with regulations.

### 6.1 Performance

- **Efficient Authentication Operations:** The service shall be optimized for low-latency processing of authentication requests. A typical login or token validation should be extremely fast (on the order of tens of milliseconds on the server side, not counting network latency) under normal load. Using lightweight components like Dapper for data access will help minimize ORM overhead <sup>5</sup>. Key database queries (such as looking up a user by username/email and verifying password) must be tuned with proper indexes (e.g., an index on the username or email field) <sup>5</sup> to ensure constant-time lookups even as the user base grows.
- **Concurrency and Throughput:** The system should handle a high number of concurrent requests. It shall support at least  $N$  logins per second and  $M$  token validations per second (where  $N$  and  $M$  are numbers to be determined by expected usage; for example,  $N=100$  logins/sec might be a baseline for a mid-sized application). The architecture (stateless processing of JWTs, connection pooling for



DB) should enable linear scaling with additional server instances. There should be no significant global locks or single-threaded bottlenecks in the application.

- **Password Hashing Performance:** Password hashing (e.g., bcrypt) is intentionally CPU-intensive for security. The system must strike a balance between security and performance by configuring the hashing complexity such that login/registration is reasonably quick (e.g., under 0.5 seconds for hash verification on typical hardware) while still strong. The chosen algorithm's work factor can be tuned based on deployment hardware. The requirement is that the service can handle concurrent hash computations (during spikes of login) without undue delay or denial of service. If needed, the system could offload hashing to background threads or use async to not block the main request thread.
- **JWT Token Generation Overhead:** JWT creation and signing should be efficient (the .NET libraries for JWT are quite optimized). The overhead of signing (HMAC SHA-256 or RSA SHA-256) must be minimal. If using RSA with private key, ensure the key size (e.g., 2048-bit) is secure but also doesn't cause unacceptable slowness. Typically, HMAC is faster and will be used unless public-key validation is required. The service must ensure that adding claims and signing the token happens quickly. Caching static token components (like if certain claims are the same for all tokens) is possible, but since each token is per user, caching won't apply much except perhaps the signing key object.
- **Database Access Performance:** All database operations (user queries, insert/update for tokens, etc.) should be optimized. Dapper produces direct SQL – the queries should be reviewed for efficiency and use of joins vs. separate queries. For example, retrieving a user and their roles could be done in a single query with a JOIN or two quick queries. The system should avoid N+1 query problems by batching where appropriate. Repositories should fetch only necessary data (e.g., not loading all users into memory when just one user is needed).
- **Resource Utilization:** The service shall be designed to use memory and CPU efficiently. Using Dapper (which does not track object state like an ORM) means lower memory usage per query. The application should avoid excessive object allocations in hot paths (like login) to reduce GC overhead. It should use asynchronous I/O for database and network calls to utilize threads effectively (especially if high concurrency is expected).
- **Performance Testing:** Although not a direct requirement to implement, it is expected that the service design allows for performance testing and tuning. Any potential slow points (like audit logging or very large data sets in queries) should be known and possibly mitigated (for instance, if audit logs grow huge, queries on them might need archival solutions or indexing). The system should degrade gracefully under load – e.g., if overwhelmed, it may start rejecting requests with proper status (like HTTP 429 Too Many Requests) or queue them, rather than deadlocking or crashing.
- **Scenarios:** The service must be performant across common scenarios: a burst of login requests in the morning (if many users log in at start of work day), rapid token validations (if a single user makes many API calls with their token, verifying JWT should not involve DB calls each time in normal mode), and bulk operations by admins (like adding 100 users to a group – that should be handled by set-based DB operations or loops with minimal overhead). The design requirements ensure these patterns are feasible.

## 6.2 Scalability

- **Horizontal Scalability:** The authentication service shall be designed to scale out horizontally by running multiple instances behind a load balancer. Because the application is stateless (except the shared database and token store), adding more server instances should increase capacity proportionally. No single instance should hold unique session information that others don't have. For example, a user can hit instance A for login and instance B for a subsequent token validation –

this should work seamlessly since all instances share the same persistent store and JWT validation logic (and JWTs are self-contained).

- **State Management:** To facilitate scaling, any in-memory state should be avoided or kept strictly ephemeral. Session data is stored in the database (or a distributed cache if used) so that any node can access session info. If caching is implemented (e.g., caching user roles or permissions to speed up checks), a distributed cache (like Redis) or short-lived local cache that auto-invalidate is recommended so that one node's cache doesn't become a source of inconsistency. The requirement is that the behavior of the system is the same regardless of which node handles a request.
- **Database Scalability:** The underlying database should be able to handle growth in user count and activity. The design should support moving to a stronger DB solution if needed (e.g., migrating from a single-instance DB to a cluster, or from SQL Server to a cloud-managed database with read replicas). The repository abstraction and Dapper usage will allow targeting different SQL dialects if necessary with minimal changes <sup>73</sup>. For example, if reads become heavy, read replicas could be introduced; the system could direct certain read-only queries to replicas if supported (though initially, it's not required, but the design shouldn't prevent it).
- **Throughput Scaling:** For increasing throughput needs, additional instances of the service can be launched. There should be no hard-coded limits (like fixed thread pools that can't be extended) that prevent effective use of added resources. The .NET thread pool and async model will generally handle this, but any custom resource pools should be sized dynamically or allow reconfiguration. The service should be tested up to a certain number of concurrent users (for instance, 10,000 concurrent sessions) and support scaling beyond with more hardware.
- **Stateless Authentication Token:** By using JWTs, the system offloads session validation to the token itself for most request authorizations. This means that even if the user base grows to a very large number, the overhead per request (for checking a token) does not grow per user. There's no central session lookup needed for each API call (unless we choose to check blacklist), which makes it naturally scalable for token validation. The only scalability consideration is managing the token signing verification keys (which is minor) and ensuring the token size remains reasonable (embedding thousands of permissions might inflate a token – but typically that's not done).
- **Scaling External Interfaces:** If an external provider (like Google login) is introduced and heavily used, that part might become a bottleneck if our service waits on external API calls. The design should allow concurrency for those calls as well, and perhaps use caching for things like provider public keys or user info if allowable. Also, using message queues or asynchronous flows for things like sending emails can improve perceived scalability (the user doesn't wait for the email to actually send).
- **Batch Operations:** For operations that inherently scale with data (like generating reports of all users, etc.), the system should implement paging or batch processing to avoid memory issues. For example, if an admin requests all audit logs for a year, the service might stream them or require filters rather than load millions of records at once. This ensures that even large scale data can be handled without tipping over the service.
- **Cloud and Containerization:** The service will be deployed likely in a container orchestrator (Kubernetes, Docker, etc.) which makes scaling up/down easier. The requirements for scalability include that a new instance can start and register with the load balancer quickly (stateless startup, aside from reading config). Also, if using container limits, the app should behave well when approaching resource limits (not leak memory, etc., so that scaling triggers can be reliably set).
- **Graceful Scaling:** When scaling down (removing instances), the service should support graceful shutdown (completing in-flight requests before termination) so that scaling events do not disrupt ongoing logins or other processes. .NET's host has support for graceful shutdown signals which we should honor.

In summary, the service must handle growth in number of users, number of concurrent sessions, and overall request volume by scaling horizontally and efficiently managing state. The architecture (stateless JWT auth, central DB, and DDD decoupling) supports this by design.

## 6.3 Security

- **Password Security:** All user passwords shall be stored as securely hashed values with salts, as stated earlier. The system must use a strong hashing algorithm (bcrypt, PBKDF2, Argon2 or similar) with an appropriate cost factor <sup>15</sup>. The implementation will utilize an `IPasswordHasher` service which can be updated to newer algorithms if needed (e.g., if in future Argon2 is adopted, we can implement it behind the same interface). No plaintext or reversible encryption of passwords is permitted. Additionally, when users change their password, the new hash replaces the old; the system should not keep history of old passwords unless required (if it does, those too must be hashed).
- **Token Security:** JWT tokens will be signed with a secret or key that is securely stored. If using symmetric signing (HMAC), the secret key must be long and random (at least 256-bit) and stored in a configuration that is not exposed (like an environment variable or secure config store). If using asymmetric keys (RSA/ECDSA), the private key must be protected (in a keystore or vault) and only the public key would be distributed for verification. Tokens should have a relatively short expiration (e.g., 15 minutes to 1 hour for access tokens) to limit exposure if stolen <sup>46</sup>. Optionally, refresh tokens (longer-lived, stored server-side or as secure cookies) can be used to allow obtaining new access tokens – if implemented, refresh tokens must be securely stored (httpOnly cookies or encrypted storage on client, and tracked on server side to allow revocation).
- **Transport Security:** The service shall enforce the use of HTTPS for all network communication. In a production environment, the API endpoints must refuse plain HTTP (or be behind a load balancer that terminates HTTP and only talks HTTPS to clients). This ensures that credentials, tokens, and other sensitive data are encrypted in transit. Additionally, any integration with external providers (e.g., OAuth flows) must use secure redirect URIs and HTTPS for callback endpoints.
- **Input Validation and Sanitization:** All inputs to the service (API payloads, headers, query parameters) shall be validated to prevent injection attacks and ensure they conform to expected formats. Since Dapper uses parameterized queries when mapping objects, the risk of SQL injection is mitigated, but we must ensure to always use parameters for any dynamic SQL. User-supplied data (like username, email) should be handled carefully in queries. Also, the service should enforce size limits on inputs (e.g., username max length) to prevent resource exhaustion or buffer issues.
- **Authentication Attempts Security:** Measures such as account lockout after excessive failed logins (as described in functional requirements) must be implemented to deter brute-force attacks. Additionally, the system should consider **rate limiting** at the API level for certain endpoints (e.g., the login endpoint) to further mitigate automated attack attempts. For example, throttle repeated login attempts from the same IP or account. This might be implemented outside the service (like at an API gateway or using a middleware), but it's a security requirement to address.
- **Session Hijacking and Replay Prevention:** The system's use of JWT means if a token is stolen, it could be used until expiration. To mitigate this, the service issues short-lived tokens and has the ability to revoke tokens via the session store. The requirement is to have a **logout** and possibly a "force logout all sessions" feature so if a user reports theft, tokens can be invalidated. In environments requiring higher security, consider using refresh tokens with rotation (so that if an access token is stolen, it expires quickly and without the refresh token it can't be renewed, and

refresh tokens would be invalidated if used twice). While not explicitly in initial scope, the design should be compatible with adding refresh token logic.

- **Sensitive Data Handling:** Personal data like email addresses, names (if collected), and audit log details (IP addresses) should be protected. In the database, at minimum, access to this data is controlled via proper authentication and authorization in the service. Optionally, encryption at rest can be applied on the database level for certain columns (some systems encrypt emails or personal info in the DB). If required by compliance or company policy, the system should allow such encryption (this might be transparent to the app if using database TDE or column encryption). The requirement is that we acknowledge and plan for protecting data at rest appropriately.
- **Administrative Security:** Administrative endpoints (role management, viewing logs, etc.) must require admin authorization. The system will use role-based checks (e.g., user's JWT must contain a role like "Admin" or a specific permission claim) to guard these endpoints. We must ensure that normal users cannot escalate privileges by calling admin APIs. This also means the initial setup needs a secure way to have an admin user (either one is pre-seeded or there's a secure bootstrap process to create the first admin).
- **Logging and Monitoring for Security:** The service shall log security-related events (as covered in audit logging) and should integrate with monitoring to alert on suspicious patterns, such as repeated failed logins or sudden spikes in activity. This requirement means that logs should be fed to a system or can be queried in such a way to facilitate intrusion detection. Although actual monitoring is operational, the application should at least make relevant data available (which it does via audit logs).
- **Protection Against Common Vulnerabilities:** The implementation must follow OWASP ASVS and Top 10 guidelines to avoid common vulnerabilities: e.g., prevent XSS (though primarily an API, still must not include any scriptable content in responses), prevent CSRF on state-changing actions (if cookies were used, would need CSRF tokens; with pure JWT in Authorization header, CSRF is less of an issue as cookies are not used for auth by default), ensure secure deserialization (JWT libraries handle token parsing securely), etc. In short, the service should be built with secure coding practices.
- **Security Testing:** The system shall be amenable to penetration testing and code review. Design choices like using a limited tech stack (no unnecessary open ports or services) and minimal external dependencies reduce the attack surface. Also, dependency management (see section 7.2) means keeping libraries up to date to patch any known vulnerabilities.
- **Secret Management:** All secrets (JWT signing keys, database credentials, OAuth client secrets, etc.) must be stored and managed securely. The service should support loading these from secure configuration (environment variables, key vaults) rather than hardcoding. It must also handle rotation of secrets (for example, the JWT signing key might be rotated periodically; the design should allow switching keys and maybe keeping previous key to validate older tokens for a short window).
- **Compliance-Related Security:** For GDPR and similar, security of user data is a must. Encryption of personal data fields at rest, pseudonymization in logs if needed, and strict access control to data (only authorized calls return user info, etc.) all contribute to regulatory compliance.

In essence, security is woven throughout the requirements: from how we store data, generate tokens, to how we design the API endpoints. Each mechanism (password hashing, token signing, input validation, etc.) is chosen to mitigate specific threats. The service must maintain a strong security posture since it is the gatekeeper for all user accounts in the system.

## 6.4 Availability

- **High Availability Architecture:** The authentication service is critical infrastructure and should be highly available. The system shall be designed such that there is no single point of failure. Running multiple instances as mentioned (scalability) also serves availability – if one instance/node goes down, others continue serving requests. The load balancer or service discovery mechanism should detect instance health and automatically route traffic only to healthy instances.
- **Redundancy:** All components of the system should have redundancy or failover strategies. For example, the database is a single point in many setups; to ensure availability, a failover replica or cluster (like SQL Server failover cluster or a cloud DB with multi-AZ deployment) is recommended. The requirements include planning for database backups and failover (see Backup and Recovery section) so that if the primary DB crashes, a standby can take over with minimal downtime. Similarly, if using a cache or external services (like an email server), those should ideally have redundant endpoints or be robust.
- **Uptime Target:** The service should target an uptime consistent with a high-availability system (for instance, 99.9% or above). This translates to minimal allowed downtime per month. To achieve this, unplanned downtime (crashes, etc.) must be rare and short, and planned downtime (deployments, upgrades) should be handled without full service interruption (e.g., using rolling deployments or maintenance windows late off-hours).
- **Graceful Degradation:** In case of partial failures, the system should degrade gracefully. For example, if the email service is down, that should not make the entire auth service unusable; instead, registration can still create the user and perhaps queue the email for later or return a message to retry verification. If the audit log database writes fail (perhaps DB temporarily read-only), the system should still allow logins (as noted earlier, audit failures shouldn't break main flows) <sup>67</sup>, maybe logging to an in-memory fallback or at least not crashing. This way, a component failure doesn't cascade into total outage.
- **Health Checks:** The service shall include a health check endpoint (e.g., `/health`) that can be used by orchestrators or load balancers to verify the service is up and functioning. This endpoint might perform basic checks such as verifying it can connect to the database and perhaps that necessary dependencies (like a required config or key) are present. If any critical dependency is down (e.g., DB unreachable), the health check could report unhealthy, prompting orchestrators to take appropriate action (like not routing traffic to that instance or trying a restart).
- **Auto-Recovery:** The system should be configured so that if a process crashes, it's automatically restarted (e.g., if running in Kubernetes, a CrashLoopBackoff will restart the pod; or in a Windows service, set to auto-restart). The application should be stateless enough that a restart only affects in-flight requests. On restart, it should quickly re-establish connections (like to DB) and resume service.
- **Maintenance and Upgrades:** For deploying new versions, the system should allow rolling updates where instances can be taken offline one at a time, updated, and brought back, without a total outage. This requires that the new version is backward compatible with the old (especially regarding the database schema or token formats, etc.) or that downtime is coordinated if a breaking change is needed. Minimizing such breaking changes is part of high availability planning.
- **Capacity Planning:** Availability can suffer if the system is overwhelmed beyond its capacity. Therefore, capacity planning (ensuring we have enough instances, DB size, network bandwidth for peak loads) is a part of maintaining availability. The system should be monitored for usage and scaled up proactively before hitting limits. Also, use of connection pooling for the DB ensures we don't exhaust connections under load. The app should guard against resource exhaustion – e.g., limit maximum concurrent DB calls if needed to avoid DB overload; such protective limits ensure the service might throttle rather than crash in extreme situations.

- **Disaster Recovery:** While Backup/Recovery deals with data, availability also means having a plan for disasters. The requirements implicitly include that backups are in place, but also that there is a strategy to deploy the service in an alternate environment if the primary environment fails (for instance, if running in one data center or cloud region, consider the ability to spin up in another region). This might involve infrastructure decisions beyond the app itself, but the app should not have any hard-coded dependency on a single location (e.g., all config should be portable).
- **Monitoring and Alerting:** To maintain high availability, the service should be monitored (see Logging/Monitoring section). Alerts should be set for conditions like high error rates, instance down, or DB connection failures so that ops teams can respond immediately. While not a direct requirement on the software's functionality, it's a requirement that the software emit the necessary metrics and logs to support such monitoring.

Overall, the system is expected to be available virtually all the time except for rare maintenance events, and even those should be as transparent as possible to end users. By eliminating single points of failure and enabling quick recovery, the service will meet the availability demands of a critical auth system.

## 6.5 Logging and Monitoring

- **Application Logging:** Beyond the audit log (which is a domain-level log of security events), the application shall produce operational logs for debugging and monitoring. These logs include information on startup, configuration loaded, errors/exceptions encountered, and other runtime info. The logging should use a structured format (JSON or clear text with fields) so that log aggregation systems can parse it. For example, use .NET's built-in logging framework with providers for console, files, or centralized logging.
- **Error Logging:** All errors, especially exceptions that lead to 5xx responses or unexpected behavior, shall be logged with sufficient detail (stack traces, error messages). Sensitive info (like passwords or personal data) must not be included in error logs. But context like which user ID had an issue (if applicable), what operation was happening, etc., should be logged to assist debugging. The system should catch unhandled exceptions at a global level (to avoid crashes) and log them, ideally sending a generic error response to the client while logging internals on the server.
- **Request Tracing:** It is beneficial for monitoring to trace requests. The service should log incoming requests and their outcomes (at least for key endpoints). For example, logging each login attempt with outcome (this might overlap with audit log for login events). A correlation ID can be used (if one is provided by client or generated at gateway) to trace a request through logs. This helps in troubleshooting performance issues or errors by following a request's path.
- **Metrics Collection:** The service should expose or collect key metrics for monitoring systems. Important metrics include: number of logins per minute, number of failed logins, number of active sessions, response times for requests, CPU and memory usage, DB query timings, etc. Many of these can be obtained via infrastructure (APM tools, etc.), but the service can integrate with monitoring libraries (like AppMetrics or Application Insights in .NET) to emit custom metrics. For example, a metric for "Token Issued Count" or "Password Reset Requests Count" could be incremented.
- **Health and Heartbeat:** As mentioned in availability, a health check endpoint is part of monitoring. The service might also have a lightweight internal heartbeat log or metric (like logging "Service started" and maybe a periodic "I am alive" if needed, though usually external monitors suffice).
- **Monitoring Systems Integration:** The system should be easy to integrate with common monitoring stacks (ELK/Elastic Stack, Prometheus + Grafana, Azure AppInsights, etc.). This usually means using standard protocols or formats: e.g., logs to stdout (for container logging) in JSON, metrics via an

endpoint or integration library. The requirement is that the dev team can configure these without code changes (just by enabling the appropriate logging provider or metric exporter).

- **Alerting:** While the URS doesn't implement alerting, it should generate the data that would drive alerts. For instance, logs of multiple failed login attempts for the same user could be picked up by a SIEM to alert potential brute force attack. High latency in responses (which could be logged or measured) might trigger an alert to investigate performance. The service should thus log unusual conditions, such as "DB connection lost, retrying" or "External provider X is slow/unresponsive", so that ops can react.
- **Log Privacy:** Ensure that no sensitive personal data or secrets are written to logs. This is crucial for compliance. For example, do not log full JWT tokens or password reset tokens (they are like credentials). If needed for debugging, log maybe the first few characters or a hash to identify it without exposing it. Do not log passwords ever (even in validation failures). Also, avoid logging PII in debug logs unless necessary; if logged, consider masking or hashing it. Logging guidelines should be documented and followed in implementation. This ties into GDPR compliance where even logs must not store personal data without reason <sup>74</sup>.
- **Audit vs Operational Logs:** Maintain a clear separation between audit logs (security events stored in the database) and operational logs (system logs perhaps stored in files or log management system). This separation is important so that the audit log remains pristine for security reviews, while verbose debug info is kept elsewhere. The URS requires both exist.
- **Testing and Validation of Logging:** It's a requirement that during development, the team should verify that logs contain useful info and that metrics are recorded correctly. For example, simulate a user flow and ensure it produces the expected audit entries and no unexpected sensitive data in the general logs. This ensures monitoring will have the needed coverage.
- **Monitoring External Dependencies:** The service should also monitor its interactions with external systems. For instance, if it calls an SMTP server to send mail, and that fails or is slow, log that event. If an external auth provider call fails or returns errors, log those details. This helps in diagnosing integration issues (e.g., "Google API returned error X, user could not log in via Google"). Possibly include performance timing of external calls in logs or metrics to catch slowness.
- **Resource Monitoring:** The application could optionally provide endpoints or use existing ones for resource usage (like .NET's event counters or diagnostics endpoints for memory, threads, etc.). This can help detect memory leaks or high CPU loops early via monitoring.

In short, the system needs robust logging of all key events and errors, and it must emit metrics that allow live monitoring of its health and usage. Combined with audit logging, this provides full visibility into both the security and performance of the service.

## 6.6 Compliance (GDPR, Data Retention)

- **Data Minimization:** The service shall collect and retain only the personal data necessary for its function (per GDPR's data minimization principle) <sup>74</sup>. For example, to create an account, we need an email and password; we should not require additional personal details that are not needed for authentication. Any additional user profile data (address, etc.) is beyond the auth scope and not stored here. Within logs, personal data is limited to what's needed (e.g., recording user ID or email in an audit log for security context) and not more.
- **Consent and Lawful Basis:** The system will operate typically under the lawful basis of "legitimate interest" or "contract" for processing user data (since users create accounts to use a service). If any aspect requires user consent (e.g., receiving emails), that must be handled at the application level,

not necessarily within this auth service. However, the service should be built such that it can accommodate a user's request related to data (like deletion), as described below.

- **Right to Access:** Users (or administrators on their behalf) should be able to retrieve the personal data stored about them. The service shall provide means to get a user's profile information (email, username, and any linked data like roles or external logins) via an API (likely an existing get user endpoint suffices). Audit logs pertaining to the user could be considered personal data (especially login IPs). While not normally exposed to end-users, an admin should be able to extract a report of a user's activity if needed to satisfy a GDPR data access request. The system should organize data in a way that this is feasible (for instance, audit logs can be filtered by user ID).
- **Right to Erasure (Deletion):** The system shall support deletion or anonymization of a user's personal data upon request, as long as it doesn't conflict with other legal obligations. This means an admin should be able to delete a user account (via a "delete user" function). Deletion should remove personal identifying information from the auth database: user record (username, email, hash, etc.) and associated tokens and links. However, logs present a complication: audit logs and other historical records might need to be retained for security/legal reasons even after an account is deleted, but GDPR would suggest we either anonymize them or justify retention. A reasonable approach is: when a user is deleted, we can anonymize references in audit logs (e.g., replace user ID with a pseudonym or random ID in those records) unless those logs can be legitimately kept identifying (security interests can allow some retention). The requirement is that we have a documented approach to user deletion handling to be GDPR-compliant.
- **Data Retention Policy:** The service shall implement data retention limits for various categories of data:
- **Audit Logs / Security Logs:** Retain for a defined period (for example, 1 year or 2 years for compliance) <sup>68</sup>. After that period, logs should be archived or deleted. The system should support either an automated purge of old log entries or an archival mechanism (perhaps moving to a different table or offloading to a file) to ensure old personal data doesn't linger indefinitely.
- **Inactive Accounts:** Decide on a policy for accounts that are inactive for a long time (e.g., not logged in for 2 years). Possibly, the system may flag or remove such accounts to reduce holding data unnecessarily. This is not a must-have requirement, but if business chooses, the system should allow querying last login timestamps (which would be in audit logs or a user field) to facilitate this.
- **Expired Tokens:** Password reset tokens, email verification tokens, and similar ephemeral data should be short-lived and the system shall automatically purge them after expiration. For example, a daily job can delete all reset tokens older than, say, 7 days (way past their expiration). This avoids accumulation of personal data (these tokens might not be sensitive once expired, but they serve no purpose either). Similarly, expired session records could be removed or archived periodically.
- **Encryption and Security of Personal Data:** To comply with regulations, the system should implement appropriate technical measures, which include encryption of data at rest where suitable. If the database supports encryption (TDE) or we can encrypt certain fields (like storing password hashes is one form of protecting that data, though it's one-way encryption), we should enable those. Backup files of the database should also be encrypted because they contain personal data. Access to logs and databases must be restricted to authorized personnel, which is more of an operational policy but enabled by our design of roles and separation.
- **Breach Notification Support:** In case of a security breach (for instance, detection that the database was compromised or an admin's account was misused), our audit logs and monitoring should allow quick assessment of what data might have been accessed. The system should log access to admin functions as well (so we know if someone enumerated user data, etc.). This supports breach analysis and notification duties by providing the needed info.



- **Compliance with Other Standards:** If the system needs to comply with other frameworks (HIPAA, PCI DSS, etc., depending on use), additional requirements may come (like password policies, multi-factor auth for certain data, etc.). At minimum, our design aligns with many best practices. For example, PCI DSS would require strong password rules and possibly not storing certain sensitive info. Our system doesn't store credit card info, so that's fine. For HIPAA (health data), audit logging is a requirement which we have. So generally, the architecture should be adaptable to those needs if they arise.
- **Consent for Cookies/Tracking:** Our service itself doesn't use cookies (unless implementing refresh tokens in cookies in future). If it did, compliance would require secure attributes on cookies and possibly user consent banners for tracking. But as an API, this is not directly applicable. However, if our tokens are used in a web context, ensure the app using them does so securely (this might be documented externally).
- **Internationalization of Data Storage:** GDPR also has rules about transferring personal data outside EU. If our deployment is global, we must consider where user data is stored. This might be a decision to host EU user data in EU data centers. The service design should allow deployment in multiple regions as needed. It's not an intrinsic requirement, but it supports compliance if needed (the code doesn't prevent multi-region setups or splitting user data by region).
- **Policy and Documentation:** While not a software feature, it's worth stating: all these compliance measures should be documented (data flow diagrams, what data is stored, for how long, etc.) to demonstrate compliance. Our design inherently includes many compliance-friendly features, and it will be documented for auditors if required.

In summary, the service will adhere to privacy by design principles: only handle necessary data, secure it strongly, keep it only as long as needed, and respect user rights regarding their data. By following these requirements, the system will facilitate compliance with GDPR and similar regulations, avoiding hefty penalties and ensuring user trust.

## 7. Operational Requirements

This section details requirements related to deploying, managing, and maintaining the Authentication service in a production environment. It covers the architecture of deployment, dependency management, backup/recovery procedures, and general maintenance considerations to ensure the system runs smoothly over its lifecycle.

### 7.1 Deployment Architecture

- **Service Packaging:** The Authentication service shall be packaged and deployable as a self-contained unit (for example, a Docker container image or a standalone application) that includes the .NET 10 runtime and all necessary libraries. The deployment artifact should be consistent across environments (Dev, Test, Production) to avoid configuration drift.
- **Microservice Setup:** The service will operate as a microservice with a well-defined API. It will be deployed behind an API gateway or load balancer that routes requests to it. The service will listen on a configurable port for HTTP(S) requests. There should be no user interface in this service – only API endpoints – which means it can be deployed on a server or container without any GUI components.
- **Separation of Concerns:** The deployment should isolate this auth service from other services. It may reside in its own container or process. However, it will interact with other components like the database and possibly an email SMTP server or external IdP endpoints. These dependencies will be

referenced via configuration (e.g., DB connection string, SMTP server address, etc.), not hard-coded, allowing environment-specific endpoints in each deployment.

- **Database Deployment:** A relational database (such as Microsoft SQL Server, PostgreSQL, etc., as decided) is required for the service. The deployment architecture must include provisioning this database and applying the necessary schema (tables for users, roles, groups, tokens, audit logs, etc.). Schema migrations should be managed (via scripts or a tool) and versioned to deploy updates as the application evolves. The service, upon startup, might verify the presence of required tables or run migrations (depending on the strategy chosen).
- **Networking and Security:** The service will be deployed in a secure network environment. In a cloud setting, it might be in a virtual network or Kubernetes cluster. It should only expose its API port to necessary clients (e.g., front-end or other services) and not to the public internet directly (unless we intentionally make it public for, say, a mobile app to hit). Firewalls or ingress rules should restrict access appropriately (e.g., the public might access only the specific endpoints like login, whereas admin endpoints could be further firewalled to internal IPs or VPN). If applicable, mutual TLS or network ACLs can protect internal communications (like between the service and database).
- **Configuration Management:** Deployment must handle configuration values for different environments. This includes secrets like DB credentials, JWT signing keys, email server creds, OAuth client secrets, etc. The requirement is to use a secure method of injecting config, such as environment variables (with Docker/K8s secrets) or a configuration service/vault. The service should be configured via external settings, not requiring code changes per environment. It should support a hierarchical config (like `appsettings.Development.json` vs `Production` or environment variables overriding defaults) to easily adapt to each deployment.
- **Scalability & Load Balancing:** The architecture will include a load balancer if multiple instances of the service are deployed. For example, in Kubernetes, a Service or Ingress will distribute traffic to pods. The service itself doesn't maintain session state (thanks to JWT statelessness), so simple round-robin or least-connections LB will work. Sticky sessions are not required. The design will allow adding or removing instances dynamically (manual or auto-scaling). The load balancer or orchestrator should also perform health checks (the service should have a `/health` endpoint as noted) and only route to healthy instances.
- **Logging and Monitoring Integration:** The deployed architecture should integrate with logging and monitoring solutions. This might mean mounting a volume or endpoint for logs, or using a sidecar (like Fluent Bit) to ship logs to an ELK stack. It can also mean enabling Application Insights or Prometheus exporters in the container. The operational requirement is to ensure the deployed service's logs and metrics can be collected by the organization's monitoring infrastructure. The deployment scripts/config should set up any needed agents or config to achieve this.
- **Email and External Services:** If the service needs to send emails (for verification/reset), the deployment must ensure network connectivity to an SMTP server or email API. That endpoint (hostname/port, credentials) is part of config. Similarly, if enabling Google or other external logins, the service will need internet access to call those providers (or through a proxy if corporate policy). The architecture should account for these outbound calls (e.g., open firewall for outbound port 443 to known IdP domains).
- **Redundancy and Failover:** The deployment should ideally be across multiple availability zones or data centers to avoid a single point of failure. For instance, run instances in two separate AZs with the load balancer across them, and use a multi-AZ database. This way, if one zone goes down, the service in the other zone can continue. This ties into the availability requirement; it's more of an infrastructure consideration but critical for operations.

- **Resource Allocation:** When deploying, allocate appropriate CPU and memory resources to the service. .NET 10 (Core) is quite efficient; even a small container (e.g., 2 vCPU, 4GB RAM) might handle thousands of auth ops per second, but this depends on load. The requirement is that ops can tune these resource limits and requests in container orchestrators to ensure the service performs well but also doesn't starve or hog resources on a shared node.
- **Deployment Automation:** There should be an automated process (CI/CD pipeline) for deploying new versions of the service. This ensures consistency and repeatability. The pipeline can run tests, then build the container, then push to environment. Part of operational requirements is that deploying a new version can be done with minimal downtime (rolling update). If database migrations are needed, they should be applied carefully (backwards-compatible changes where possible, or brief maintenance windows if not). The team should plan deployment steps to avoid breaking running tokens (e.g., if changing token format or claims, try to support old and new for a transition period).
- **Rollback Plan:** The deployment architecture must support rolling back to a previous version if a problem is discovered. This means keeping the last known good container image available and possibly maintaining backward compatibility on database changes or having a quick way to restore the previous DB state. This is more of an ops procedure, but the application design can help by, for example, using additive migrations (so an older version still runs with a superset schema, albeit with maybe unused columns).

## 7.2 Dependency Management

- **Internal Dependencies (Layered Architecture):** The project will be structured into layers (e.g., Domain, Application, Infrastructure) enforcing DDD and clean architecture boundaries. This structure shall be reflected in the solution (e.g., separate projects or folders for each layer) and dependency injection is used at runtime to wire them together. The requirement is to use a .NET Dependency Injection container (built-in or something like Autofac) to bind interfaces to implementations at startup. For instance, `IUserRepository` might be bound to `DapperUserRepository`, `ITokenService` to `JwtTokenService`, etc. This allows managing internal dependencies easily and swapping implementations if needed (for testing or changes).
- **NuGet Packages and External Libraries:** The project will rely on some key NuGet packages (for .NET 10 compatibility). These include Dapper for data access, likely `System.Data.SqlClient` or a specific DB provider, `System.IdentityModel.Tokens.Jwt` (or Microsoft's JWT library) for token handling, possibly a library for password hashing if not writing one (e.g., BCrypt.NET), and any others as needed (like send email might use `MailKit` or similar). The requirements for dependency management are:
  - All external libraries must be properly versioned and included via NuGet; no manual DLL references that are not tracked.
  - We should prefer libraries that are well-maintained and have appropriate licenses (avoid anything with incompatible license for enterprise, e.g., GPL).
  - Keep the number of dependencies minimal to reduce attack surface and maintenance (e.g., using Dapper instead of a heavier ORM is an intentional choice to minimize dependency complexity). <sup>75</sup>
- **Updating Dependencies:** There should be a process to update libraries periodically (especially for security patches). The requirement is that the development team monitors for critical updates (like a security vulnerability in the JWT library) and can quickly release a patch. The code should be written such that library updates (e.g., a major version bump in Dapper or .NET runtime) are not too hard – adherence to standard usage helps here. Using abstraction (like interfaces) also means if one library

becomes problematic, swapping it is easier (for example, if we ever replaced Dapper with EF Core or raw ADO, having repositories abstracted makes this possible with moderate effort).

- **Database Dependency:** The service depends on a SQL database. Management of this dependency includes ensuring the DB server version is compatible (e.g., if we use SQL Server 2019 features, we need that or higher in deployment). The service should handle DB connectivity issues gracefully (retry logic on transient errors perhaps, using a library like Polly or built-in retry in connection strings). It's required that a transient network glitch to DB doesn't crash the service permanently – it should be able to retry or at least stay up so that subsequent requests can succeed when DB comes back.
- **External Service Dependencies:** The service's dependency on an SMTP server or external auth providers should be loosely coupled. For instance, the email sending functionality should be abstracted behind an interface `IEmailService`. In initial development, we might have a dummy or simple SMTP implementation. In production, this could be configured to use a real SMTP or an API like SendGrid. If the email system changes, we implement a new `IEmailService` without altering core logic. Similarly, for external providers, an `IExternalAuthProviderService` per provider type could abstract the details (so adding a new provider is adding a new class implementing a known interface). The requirement is to avoid direct, scattered calls to external SDKs in the core logic – keep them contained and swappable.
- **Compatibility:** .NET 10 (if it corresponds to .NET (Core) 7 or beyond) should be a modern, stable runtime. The service code must target this and ensure compatibility with the OS environment (if running in Linux containers, all libraries must support Linux, which most pure .NET libraries do). We should also watch for Long Term Support (LTS) status of .NET 10 – presumably it's an LTS release if used in 2025. If not, plan to upgrade to the next LTS in maintenance.
- **Documentation of Dependencies:** It's an operational need to document what major dependencies are present and any special steps required for them. For example, if using BCrypt, note that it might require native libraries (some implementations do). Or if using any COM components (unlikely here) which require windows. The expectation is that our stack (Dapper, etc.) is cross-platform and container-friendly. Document any prerequisites like "requires OpenSSL libraries in container for JWT if using X509 certs" etc. The deployment scripts should ensure such prerequisites are present.
- **Third-Party Authentication Services:** If in the future the service were to integrate with something like Azure AD B2C or IdentityServer, those themselves could be considered dependencies. However, our approach is custom (explicitly not using external full IdP frameworks) <sup>75</sup>, which simplifies dependency management (we control our auth logic). That said, if integrating, treat those as external systems with well-defined APIs.
- **Ensuring Loose Coupling:** Every dependency injection mapping ensures loose coupling. The domain layer should have zero dependency on infrastructure libraries (no direct Dapper calls in entities, etc.). The application layer may depend on domain and interfaces. Infrastructure depends on both to implement. This way, for testing, we can swap infra with in-memory fakes, etc. The requirement is that there are no hardwired dependencies across layers that break this architecture. This is important for maintainability and swapping pieces.
- **Dependency Graph Simplicity:** The final deployed service should have a clear dependency graph: AuthService -> Database (SQL), Email SMTP, External IdPs. It should not require other internal services running (like if it depended on a caching service or message queue, we would list those, but currently not required). Minimizing such dependencies ensures if something goes down, there's less to break. If we do use a cache or message queue (maybe in future for audit logs or email queue), those would be additional dependencies to manage (with their own failover). At present, keep it minimal: DB and optional SMTP/IdP connectivity.

## 7.3 Backup and Recovery

- **Database Backup:** The user and identity data stored in the database is crucial and must be backed up regularly. The operations team shall establish an automated backup schedule for the authentication database. Typically, a full backup daily with incremental backups or transaction log backups hourly (if point-in-time recovery is needed) is recommended. The requirement is that it must be possible to restore the database to at most the last 24 hours (or whatever RPO – Recovery Point Objective – the business defines, possibly even shorter if needed). For example, if something goes wrong at 3 PM, we can restore to a backup from no earlier than 2 PM (1 hour data loss at most).
- **Secure Backup Storage:** Backups shall be stored securely, ideally encrypted, and in a separate location (offsite or at least off the primary server). This is to protect against scenarios like server failure or ransomware. Only authorized personnel should have access to backups, since they contain sensitive user data (including password hashes, personal info). The backup process itself should be monitored for success/failure.
- **Recovery Process:** A documented recovery procedure must exist. In the event of data loss or corruption (say the database is compromised or an admin accidentally deletes data), the team should know how to restore from backup. The requirement is that the service can be pointed to a restored database and continue functioning. This implies that the application should handle a scenario where data is restored to an earlier state – e.g., if some audit log or token entries are out of sync with what was in memory (though since stateless, memory isn't an issue except perhaps JWTs that were issued after the backup point might be considered invalid if user records not present; but tokens self-contained will still validate as long as signing key didn't change). The system should allow re-issuing tokens or redoing whatever was done in lost period.
- **Disaster Recovery and Failover:** Beyond backups, ideally the deployment would have a hot standby or replica of the database that can take over if the primary fails (this provides high availability rather than a full restore). If using a cloud-managed DB, enabling geo-replication or at least same-region replicas is suggested. The requirement from an operational view is to minimize downtime during a recovery. If a DB server goes down, a failover to a replica should occur automatically or with minimal manual intervention. The application should be configured (or able to be reconfigured quickly) to point to the new primary. Using a virtual IP or connection string that encapsulates failover helps.
- **Consistency and Transactions:** During runtime, critical operations (like registering user and creating token) are done in transactions <sup>38</sup> to maintain consistency. This reduces the chance of partial data that might need manual cleanup. In a crash scenario, either the transaction is rolled back or committed fully, so we won't have half-created users. The requirement is that this principle be applied to all multi-step data modifications (and indeed we have planned it accordingly). This simplifies recovery because data integrity is maintained.
- **Configuration and Key Backup:** Not only the database, but configuration and keys should be backed up. For instance, the JWT signing key (if symmetric) or certificate needs backup. If we lost that key (and it wasn't stored in source control for security reasons), all tokens would become untrustable unless we intentionally rotate keys. So, the signing keys (or any encryption keys used) should be stored in a secure vault that is itself backed up or recoverable. Similarly, any custom configuration like lists of roles/permissions if not easily reproducible should be documented or backed up (though these largely reside in the DB or code).
- **Testing Recovery:** It's an operational requirement that backup and restore procedures are tested periodically. For example, performing a drill where a backup is restored to a staging environment to ensure it works, and measuring how long it takes. The system should ideally be able to recover within a target RTO (Recovery Time Objective), say within X hours of a catastrophic failure. If the database is small (which it often is for an auth DB relative to something like a data warehouse),

backups and restores are quick (minutes). But if audit logs accumulate heavily, the DB could grow, so monitoring size and maybe archiving old logs to keep DB size manageable could also assist in faster recovery.

- **Logging after Recovery:** After a restore, the system might have to deal with certain stale data issues. For example, say an account was created and then lost in the backup gap; a user might try to register again and find the email “already taken” if the account partially existed. Or vice versa, maybe they think they have an account but it’s lost. The service should handle such gracefully (the simplest way is treat it as new or ask for re-registration). Auditors will want to know if data was lost – the team should be able to check what accounts or actions occurred in the lost period (maybe using application logs if they were separate from DB, as they might have captured events even if DB was lost). Therefore, ensure application logs are also archived; they could help fill gaps.
- **Version Compatibility:** If restoring a database from an older backup, ensure the running service is compatible with that schema version. If we deployed a new version and then had to restore a backup from before that deployment (maybe the deployment had a bug and we roll back code but also have to restore data), the data and code need to align. This is more of a planning detail: either avoid destructive schema changes or ensure we have backward compatibility scripts.
- **User Session Recovery:** In a restore scenario, JWTs that were issued after the backup point will effectively reference user states that might not exist in the restored DB (like a user who registered then got lost). Those tokens would fail to validate the user (if our service tries to find a user record by ID during some operations). However, if tokens are self-contained, other services might accept them until they expire, even if our DB doesn’t know that user (which is tricky). Perhaps it’s rare and acceptable given short token life, but it’s a consideration: after a restore, perhaps forcing all users to log in again (by invalidating all sessions) could be a safe approach. The requirement would be to inform users to re-login if a major restore occurred (operational communication). The service could facilitate that by having an easy way to invalidate sessions (e.g., change a key or set a “system epoch” after which old tokens are not accepted). This is not explicitly required but a potential strategy in severe recovery events.

## 7.4 Maintenance Considerations

- **Code Maintainability:** The codebase shall be organized and commented to facilitate ease of understanding for future maintainers. Using DDD patterns, while beneficial, can introduce complexity; thus, the code should include documentation (README, comments on critical classes) describing the intent of aggregates, services, and so on. New developers should be able to on-board by reading domain model documentation and understanding business rules encapsulated in the code.
- **Extensibility:** The system is designed for extensibility following Open/Closed principle <sup>28</sup>. Maintenance may involve adding new features like multi-factor authentication, additional external providers, or more complex permission rules. The requirement is that these enhancements can be made without significant refactoring of existing components. For instance, to add MFA, one could introduce a new entity for OTP tokens and some logic in the login flow, largely as an extension, not a rewrite. To add an external provider, as noted, we already have structures to accommodate it <sup>71</sup> <sup>29</sup>. This modular design will reduce maintenance cost as new requirements arise.
- **Issue Tracking and Bug Fixes:** Operationally, any bugs discovered should be easy to trace via logs and the structured design. For example, if a user cannot log in due to an issue, the logs and audit entries will help pinpoint if it was a password mismatch, an unverified email, etc. The code should be structured in a way that critical paths (like login, registration) are straightforward and not overly

tangled, making it easier to debug issues. Unit tests (see below) further ensure that maintenance changes don't break expected behavior.

- **Automated Testing:** A comprehensive suite of automated tests (unit tests for domain logic, integration tests for repository and service flows) shall be part of the project. This is essential for maintenance: when changes are made or libraries are updated, tests will catch regressions. For example, tests for password hashing verification, JWT generation, role assignment logic, etc., must be included. The requirement is that maintainers run these tests as part of CI pipeline for every change. This provides confidence that adding a new feature or refactoring won't inadvertently break existing functionality.
- **Documentation and Change Logs:** Each release or update should be accompanied by release notes documenting new features, changes, or any config/db migrations. Over time, this build-up of documentation assists maintainers in understanding how the system evolved. Architectural decision records (ADRs) for major decisions (like why Dapper was chosen over EF, why roles are without permissions initially, etc.) could be maintained to provide context to future maintainers. This URS itself is a form of documentation that can be referenced in the future to recall what requirements the system was intended to meet.
- **Configuration Maintenance:** The system will likely grow configuration options (like password policy parameters, token expiration times, etc.). It's important that these are externalized so they can be tuned without code changes. Maintainers should ensure that new features come with config toggles/parameters as needed. Also, cleaning up unused config over time is important (if a feature is removed, its config should be removed to avoid confusion). A central config file or section and a wiki explaining each setting is useful for maintenance.
- **Monitoring and Performance Maintenance:** Over time, usage patterns might change (more users, etc.). Maintainers should periodically review performance metrics and possibly re-tune aspects: e.g., increase DB pool size, add an index if queries slow down with more data, refactor a slow piece of code if profiling shows a bottleneck. The requirement is that the system should have the hooks (profiling, metrics, etc.) to allow such analysis easily. Also, as dependency versions change (e.g., .NET 10 to .NET 11), re-profiling might be needed to catch any new performance issues.
- **Dependency Updates and Patches:** A maintenance routine should be established for updating dependencies and the .NET runtime. For a secure system, patching is crucial. For example, if a security flaw is found in .NET 10, upgrading to 10.x or 11 might be needed; tests should be rerun to ensure compatibility. The design's heavy use of abstraction means ideally fewer breaking changes, but maintainers should allocate time for such upgrades. Keeping the system on supported versions ensures ongoing security and support.
- **Data Maintenance:** Over time, the database may accumulate large amounts of data (especially audit logs). Maintainers need to perform data maintenance like archiving or purging old records as per retention policies. Scripts or stored procedures might be created to do this in batches (ensuring not to lock the DB heavily). The requirement is to periodically maintain the DB: re-index if fragmentation occurs, update statistics for query optimizer, and purge/archive according to schedule. The application should be robust to data cleanup (e.g., if we delete audit logs older than 1 year, it doesn't affect anything current).
- **User Support and Admin Tools:** From maintenance perspective, having some admin tools or scripts is beneficial. For instance, a script to unlock a user account if needed, or an endpoint to force password reset for a user. While not strictly required in URS, maintainers often create such tools to help customer support. The system should be structured such that adding an admin API for something is straightforward, using existing services (for example, an endpoint to "unlock user" can call user aggregate's Unlock method). This flexibility helps the team respond to operational needs without hacking the database manually.

- **Logging Maintenance:** Log files or entries, if not managed, can grow endlessly. We have retention for audit logs; for application logs, we might use log rotation (if writing to file) or have a centralized log system that expires old logs. Maintainers must ensure log rotation is configured (for file logs) so disk doesn't fill. For centralized logging, ensure indices are managed so queries remain fast. These tasks might be on DevOps, but the application should assist by categorizing logs properly (levels, etc., so one can filter to just errors, etc.).
- **Continuous Improvement:** The requirement is that the team will periodically review the architecture and refactor or improve it as needed. DDD suggests an evolving model – if new subdomains or complexity appears, the model might need adjustments. Maintainers should not be afraid to refactor internals, thanks to the test coverage and modular design. For example, if we see a better way to handle permission assignments (maybe introduce direct user-permission assignment or role-permission linking in future), the design is flexible to incorporate that by adding new relationships without breaking existing ones <sup>62</sup>.

In summary, maintenance of the Authentication service is facilitated by its DDD architecture, modularity, and thorough documentation/testing. The system is designed not only to meet current requirements but to adapt to future needs with minimal friction. Ensuring processes around the code (like CI/CD, monitoring, backups) are solid will make maintaining this service in the long run significantly easier.

## 8. Constraints and Assumptions

This section lists the known constraints (design decisions that limit how the solution is implemented) and assumptions (conditions we assume to be true or present in the environment) for the Authentication and Identity Management Service.

### Constraints:

- **Custom Implementation Constraint:** The service is built **without using heavyweight identity frameworks** like ASP.NET Identity or external IAM products <sup>75</sup>. This is a deliberate constraint to allow full control and understanding of the domain logic. It means we implement features like user stores, password hashing, token generation, etc., ourselves (with the help of libraries but not end-to-end frameworks). This provides flexibility but also requires due diligence to match industry best practices.
- **Technology Stack:** The implementation must use **.NET 10 (or the designated version)** as the server platform and C# as the programming language. It must use **Dapper** as the data access layer to interact with an SQL database, rather than an ORM like Entity Framework. This implies writing SQL queries or using Dapper's extensions to map query results to objects. The database is assumed to be a SQL relational DB (e.g., SQL Server, PostgreSQL, etc.) and not a NoSQL store, as Dapper is SQL-centric. If future requirements demanded NoSQL, that would require reworking the data layer significantly.
- **JWT for Authentication:** The service will exclusively use **JWT tokens for session/authentication management** (specifically stateless access tokens passed in HTTP headers). There is no use of server-side session state or cookies for maintaining user sessions. This constraint means any service integrating with this auth service must be capable of handling JWT validation. It also means features like immediate token revocation require additional design (like our session store concept), since JWTs by default are self-contained. The system will not use other token methods (like opaque tokens with a cache) in the initial design.



- **Domain-Driven Design and Clean Architecture:** The solution must strictly follow a layered architecture: domain (entities, value objects, domain services) at the core, an application layer (use case services, orchestrating transactions) around it, and infrastructure (DB, email, external provider integrations) on the outside. This constraint ensures no business logic is written in controllers or UI, and no infrastructure code in domain. While this improves maintainability, it adds complexity and must be adhered to; developers cannot take shortcuts (like querying the DB directly from a controller).
- **Transactional Boundaries:** All operations that change data should be done within transactions where needed, especially when multiple aggregates are affected. For example, creating a user and token must be one transaction <sup>38</sup>. This constraint ensures consistency but also means our data store must support transactions (which SQL DBs do). It also constrains scaling in that we likely keep a single database (not an eventually consistent multi-db approach in this design). So the constraint is a **single relational database** as the source of truth for all identity data.
- **Consistency over Partitioning:** Because of the transactional approach and single DB, the system prioritizes consistency (immediate consistency of writes) over global partitioning. This is acceptable given the scope (likely one user base). If we ever needed multi-region deployments with separate user stores, that'd be a new architecture (not in current scope). So a constraint is that **the service is deployed in one logical region/environment** at a time, serving its user population, with consistency guaranteed within that environment.
- **Time and Format Standards:** The service will use standard formats and protocols – e.g., JSON for API data exchange (as is typical for RESTful services). All timestamps in tokens or logs will be in a standard format (likely ISO 8601 in UTC). Passwords will be handled in normalized form (perhaps we ensure case sensitivity as desired). These are design constraints for uniformity (not likely to change, but important to note for integration).
- **No Direct Permission on User (Initial):** The initial domain design chooses not to implement direct user-to-permission assignments or role-to-permission mapping, using groups as the intermediary for fine-grained permissions <sup>62</sup>. This constraint simplifies initial implementation; roles remain just labels, and permissions must go through groups. If an organization demands role-based permissions later, we'd have to extend the model. We assume that this approach (groups for permissions) is acceptable for now, which might be true if roles are only used for broad categories and not meant to carry entitlements.
- **External Providers Not Pre-configured:** While the design allows external providers, out of the box the service might not ship with any specific provider credentials or flows configured. Setting up Google, etc., will require providing OAuth client IDs/secrets and enabling those flows. The system at release might treat external auth as an optional extension. This is a constraint in deliverables: the core system focuses on username/password; external logins need configuration and possibly some coding to fully enable (though stubs exist).
- **Scalability Limits:** The service is expected to handle a significant load, but it's constrained by the chosen components – e.g., a single SQL database can become a bottleneck beyond a certain throughput. We assume typical use cases (maybe up to millions of users, with moderate concurrent logins). If extraordinary scale (like billions of users) was needed, constraints like single relational DB might break down. But those are outside current assumed requirements. So, implicitly, a constraint is that **scaling will be vertical to a point and horizontal at the app level, but the DB remains a central node** (we can scale reads with replicas maybe, but writes all go to one master).
- **Licensing and Cost Constraints:** All chosen technologies (.NET, Dapper, etc.) are open-source or included with platform, so there is no licensing cost. This was likely intentional to avoid constraints of proprietary software. If using SQL Server, there could be licensing, but that might be covered by environment (or we could use a free alternative like PostgreSQL). There's a constraint that the

solution should remain cost-effective – e.g., using heavy commercial IAM might not be allowed given this approach.

### Assumptions:

- **Environment Assumption:** It is assumed that the runtime environment for .NET 10 is available and supported on the deployment servers or cloud. This includes having the necessary OS (Linux or Windows containers/nodes with .NET 10 support). We assume containerization is available or at least installation of .NET runtime is possible on target servers.
- **Client Application Assumption:** We assume that one or more client applications (front-end web, mobile app, or other backend services) will consume this auth service's API. Those clients will handle UI (for login, registration forms) and will manage storing and presenting JWTs (typically in an Authorization header). We assume these clients will follow security best practices (e.g., store JWT in secure storage, use HTTPS, etc.). The service itself does not have a user interface, so we rely on clients to provide user interaction.
- **Email Delivery:** We assume an email infrastructure is in place or will be provided (like an SMTP server or third-party email service) to allow the service to send verification and reset emails. In development, a dummy or log-based email sender might be used, but in production an SMTP endpoint or API (with credentials) will be configured. We also assume users have access to their email and that email is an acceptable method for verification and password reset in the system's context. (In some contexts, SMS might be used – out of scope here, but email is assumed.)
- **User Base and Admins:** We assume that the user base will have some proportion of normal end-users and a small number of administrative users who manage roles, etc. We assume that administrative actions will be secured by role and that those admins are trusted individuals. The system, by design, will likely have an initial admin user (maybe created via a script or first-run configuration) who can then create other admins or roles. We assume this initialization is handled carefully (for example, a default admin password might be set in config or an invite link given – to be decided, but we assume it's handled securely).
- **Unique Identifiers:** We assume each user will have a unique identifier (UserId) that is either an auto-increment int or a GUID. Many design references use GUIDs for globally unique IDs. We assume either is fine, but leaning towards GUID for uniqueness across environments. We also assume that usernames and emails are unique in the system as required – there won't be two users with the same email, for instance.
- **Cryptography Assumptions:** We assume the availability of strong cryptographic libraries for hashing and JWT signing. .NET provides these out of the box (BCrypt via libraries, SHA256, RSA, etc.). We assume no government or corporate policy requires using a specific hardware security module or custom crypto beyond what we plan. If HSM is required for key storage, that's a possible extension, but initial assumption is software keys are acceptable with secure storage.
- **Compliance Scope:** We assume that GDPR (and similar laws) apply because personal data (emails, possibly IP addresses) are processed. We assume no more stringent standard like PCI (since we aren't handling credit cards) or HIPAA (unless this is for a healthcare app, but not stated). So the design caters to privacy but is not certified for, say, PCI (that would need additional controls like not logging certain things at all, etc.). If such certification was needed, further requirements would be added.
- **Time Synchronization:** We assume servers where this runs have synchronized clocks (via NTP or cloud provider) because token expiration and validation depend on accurate time. If a server's clock is way off, JWT expirations might be off. So we assume a reasonably correct system time (within a few seconds of actual).

- **Users' Email Ownership:** We assume that email verification is a sufficient proof of identity for activation. i.e., we assume users have provided a legitimate email that they have access to. The system doesn't do additional verification like phone or ID check, which is fine for typical apps. If needed, multi-factor can be added later. We assume the use case doesn't demand MFA from day one.
- **Load Assumption:** We assume a certain typical load profile – e.g., if this is an internal enterprise system, maybe hundreds or thousands of users, not millions. If it's public-facing, maybe tens or hundreds of thousands. In either case, the architecture should handle it, but we assume we're not building for extreme scale like billions of users without adjusting architecture.
- **Network Access:** We assume the service will be able to communicate with the necessary services: the database on its port, possibly a caching layer if introduced, and external networks for any OAuth providers. Any firewall rules or proxies needed will be handled by infrastructure, and we design assuming generally that these communications are allowed (with appropriate security).
- **Failure Assumption:** We assume that failures will happen (DB might go down, etc.), but short of catastrophic events, the system will be functional the majority of time. In other words, we assume no constant flapping of dependencies that would reveal design issues (like if DB goes up and down constantly, any system would struggle; we assume a stable infrastructure). We design for resilience, but not for absolutely chaotic environments.
- **User Behavior:** We assume users will follow the normal processes (they will check their email to verify, they will not typically try to bypass things). Malicious behavior like brute force is mitigated by our rules, but we assume not every user is malicious. Also, we assume users keep their credentials somewhat safe; we still handle token theft etc. with logout capability, but we assume token theft is not rampant.
- **Admin Competence:** We assume those with admin access (to manage roles/permissions) understand the implications of their actions. For instance, if an admin deletes a role, they should know it might affect users. We log it, but we assume some level of training or documentation for admins using these features. The system will try to prevent obvious mistakes (like not allowing deletion of in-use role if we choose that route), but not all (we might allow it if we think admin knows what they are doing).
- **Feature Usage:** Some features might not be initially used by the client apps (for example, maybe the initial application using this service doesn't use groups/permissions at all, just roles, or vice versa). But we include them for completeness and assume they might be used eventually or by other integrators. We assume having them does not cause harm even if not actively used (just sitting in DB).

In conclusion, the above constraints and assumptions clarify the context in which this authentication service is developed. They ensure that both developers and stakeholders have a common understanding of the boundaries of the solution and the conditions believed to hold true. If any assumption proves false in practice, or constraints need to be adjusted, those will be revisited in future requirement updates.

## 9. Appendices

*This section provides additional reference material and diagrams to support the requirements specified above. These are for illustrative purposes and to aid understanding, and are not themselves requirements.*

Appendix A: Use Case Diagram (Authentication Service)

(A high-level UML Use Case diagram could be included here, showing actors such as “End User” and “Admin” and the various use cases: Register, Verify Email, Log In, Log Out, Request Password Reset, Manage Roles, etc., connected appropriately 76 30 . This diagram depicts how external actors interact with the system to achieve the functional requirements outlined.)

Appendix B: Domain Model Diagram

(This appendix could contain a UML Class diagram of the domain model: Entities like User, Role, Group, Permission, Session, AuditLog, and their relationships. For example: User -< UserRole >- Role (many-to-many), User -< UserGroup >- Group, Group -< GroupPermission >- Permission 77 78 ; and one-to-many relations like User - Session, User - PasswordResetToken, User - EmailVerificationToken, User - AuthProvider 11 14 . Each class would show key fields (as gleaned in section 4) and perhaps important methods. This diagram helps visualize the static structure of the domain.)

Appendix C: Sequence Flow for Login

(An example sequence diagram could be provided to show the flow of a login operation: The user enters credentials in a client, the client calls the Auth API, the AuthService (application layer) uses UserRepository to get user, PasswordHasher to verify password, then TokenService to create JWT, SessionRepository to save session, returns token to client 3 45 . Including the audit logging step as a note. This sequence illustrates the interactions between components for one of the core use cases.)

Appendix D: Compliance and Logging Best Practices

(This could include a brief checklist derived from GDPR and security best practices on log data handling: e.g., “Do not log passwords or raw tokens”, “Retain audit logs 1 year maximum 17 ”, “Anonymize personal data in logs if possible 79 .” It might also reference any company policy or standard that the service adheres to. This serves as a quick reference for developers and auditors alike.)

References:

The requirements and design principles in this document are based on standard practices for identity management systems and the specific architecture outlined in the source design documents 80 25 81 13 . These include guidance on DDD implementation for authentication, secure credential handling, and compliance considerations current as of 2025. All citations (denoted by [†] ) refer to sections of the design guides or external best-practice resources that informed this specification.

17 68 69 70 74 79 **GDPR Log Management: A Practical Guide for Engineers | Last9**

<https://last9.io/blog/gdpr-log-management/>

30 76 **Guide to Key Architecture and Design Diagrams for a DDD-Based Authentication Service.pdf**

[file:///file\\_00000000d200720c8583a02d9e66b6d5](file:///file_00000000d200720c8583a02d9e66b6d5)